310-2202 โครงสร้างของระบบคอมพิวเตอร์
(Computer Organization)

Topic 8: Data Structures and Subroutines

Damrongrit Setsirichok

Topic

- Subroutines
- Memory Model for Program Execution
- Stack
- Implementing Functions in C Using a Stack
  (Low level to High level)

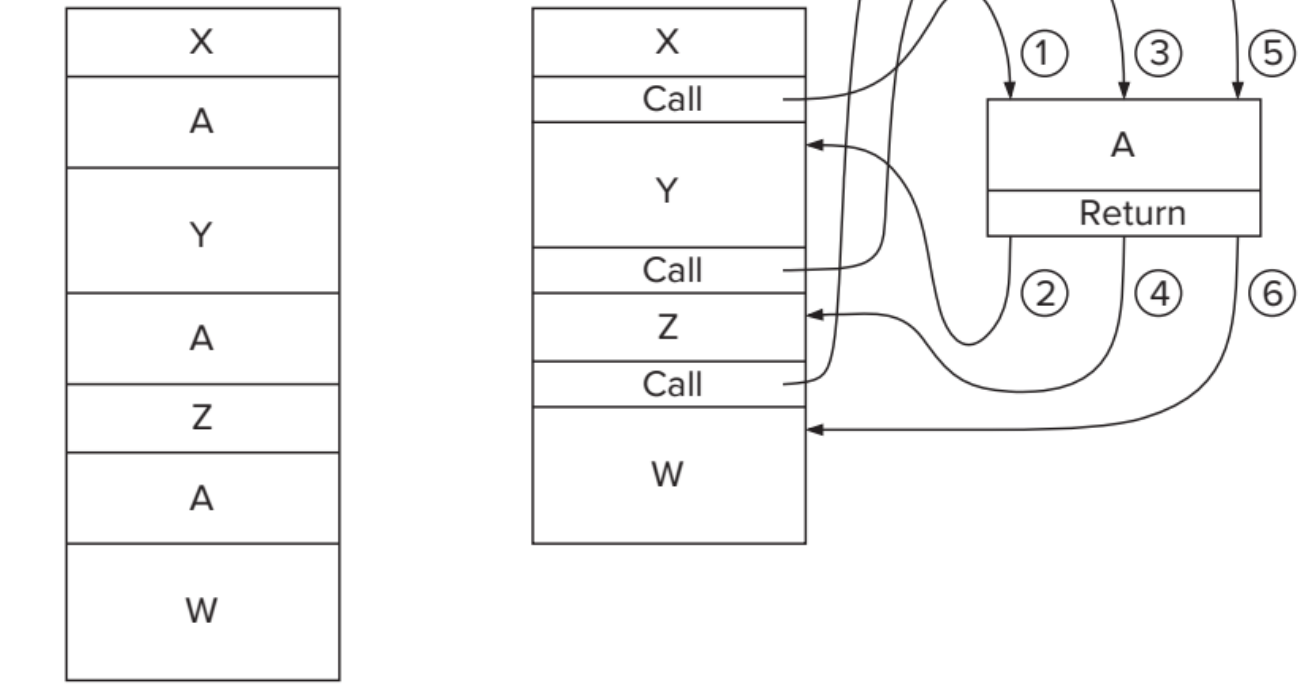
Subroutines

- A subroutine is a program fragment that. . .
  - Resides in user space (*i.e,* not in OS)
  - Performs a well-defined task
  - Is invoked (called) multiple times by a user program
  - Returns control to the calling program when finished
  - Reuse code without re-typing it (and debugging it!)
  - Divide task into parts (or among multiple programmers)
  - Use *vendor-supplied* library of useful routines that one software engineer writes a program that requires such fragments and another software engineer writes the fragments.
    - math library, square root, sine, and arctangent,etc.

A simple illustration of a part of a program

```
01 START   ST R1,SaveR1                ; Save registers needed
02         ST R2,SaveR2                ; by this routine
03         ST R3,SaveR3
04 ;
05         LD R2,Newline
06 L1      LDI R3,DSR
07         BRzp L1                     ; Loop until monitor is ready
08         STI R2,DDR                  ; Move cursor to new clean line
09 ;
0A         LEA R1,Prompt               ; Starting address of prompt string
0B Loop    LDR R0,R1,#0                ; Write the input prompt
0C         BRz Input                   ; End of prompt string
0D L2      LDI R3,DSR
0E         BRzp L2                     ; Loop until monitor is ready
0F         STI R0,DDR                  ; Write next prompt character
10         ADD R1,R1,#1                ; Increment prompt pointer
11         BRnzp Loop                  ; Get next prompt character
12 ;
13 Input   LDI R3,KBSR
14         BRzp Input                  ; Poll until a character is typed
15         LDI R0,KBDR                 ; Load input character into R0
16 L3      LDI R3,DSR
17         BRzp L3                     ; Loop until monitor is ready
18         STI R0,DDR                  ; Echo input character
19 ;
1A L4      LDI R3,DSR
1B         BRzp L4                     ; Loop until monitor is ready
1C         STI R2,DDR                  ; Move cursor to new clean line
1D         LD R1,SaveR1                ; Restore registers
1E         LD R2,SaveR2                ; to original values
1F         LD R3,SaveR3
20         JMP R7 ; Do the program's next task
```

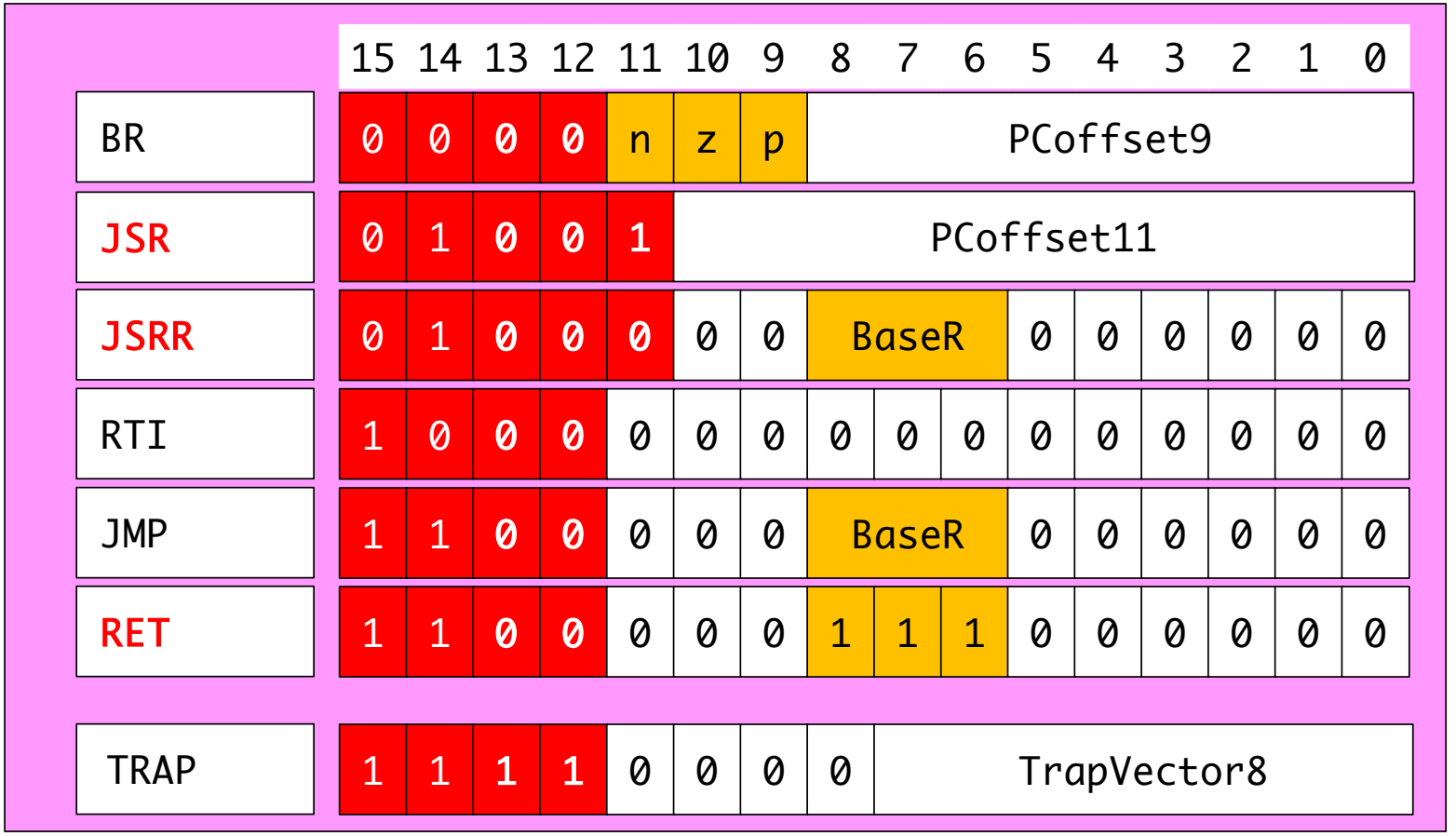
The Call/Return Mechanism



(a) Without subroutines

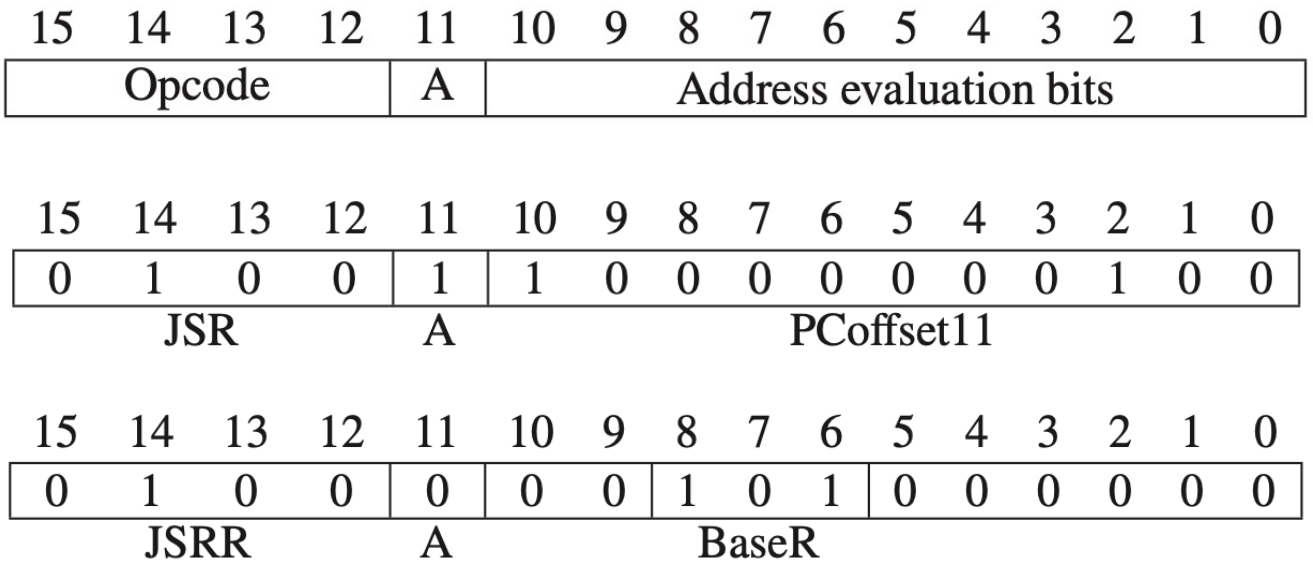
(b) With subroutines

Control Instructions for Subroutines

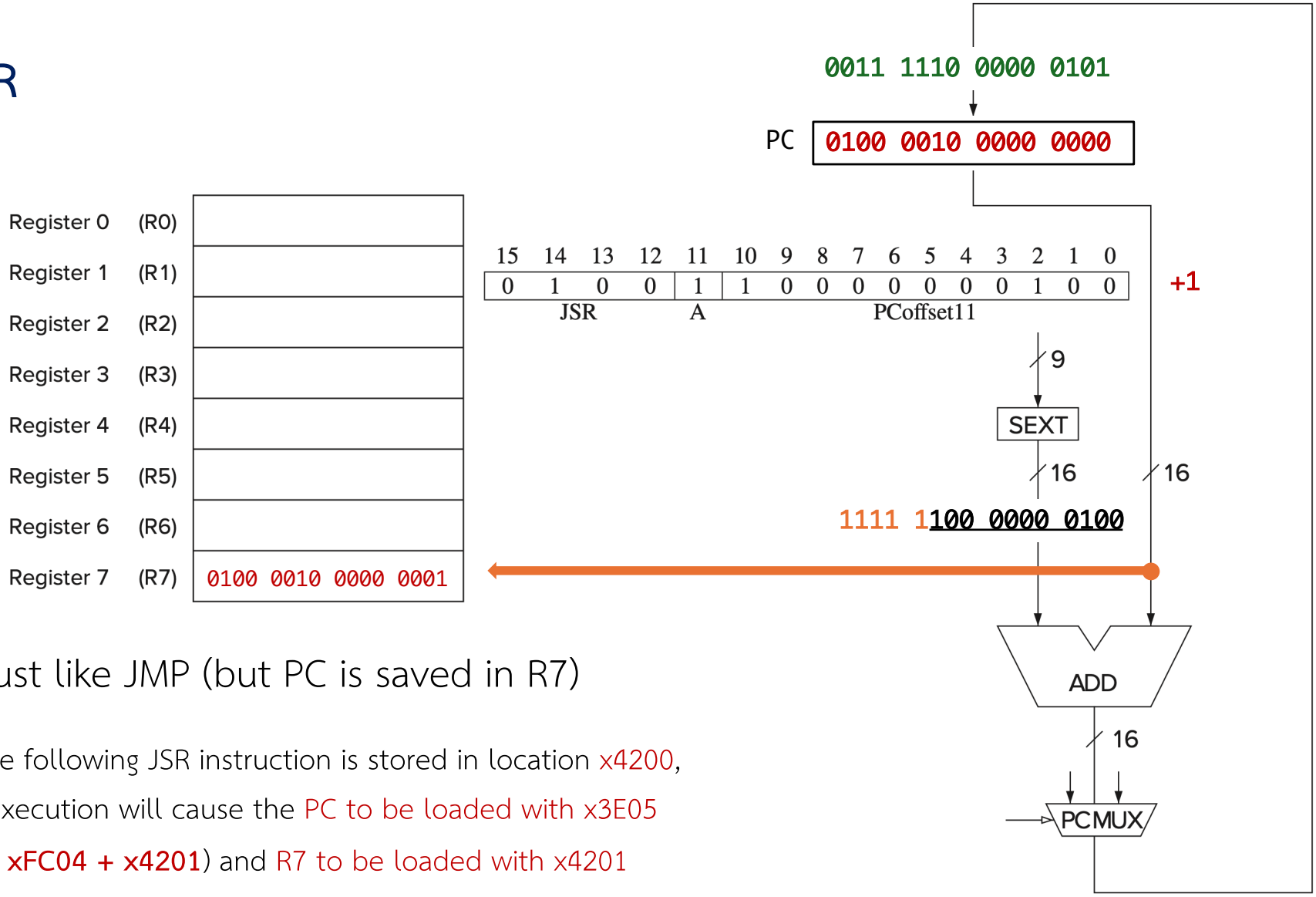


JSR(R)—The Instruction That Calls the Subroutine

- Consists of three parts



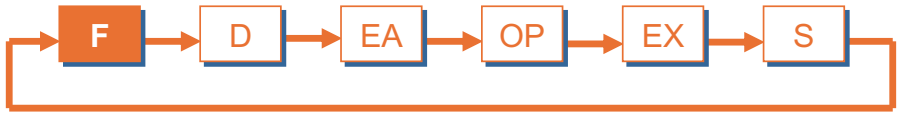
JSR

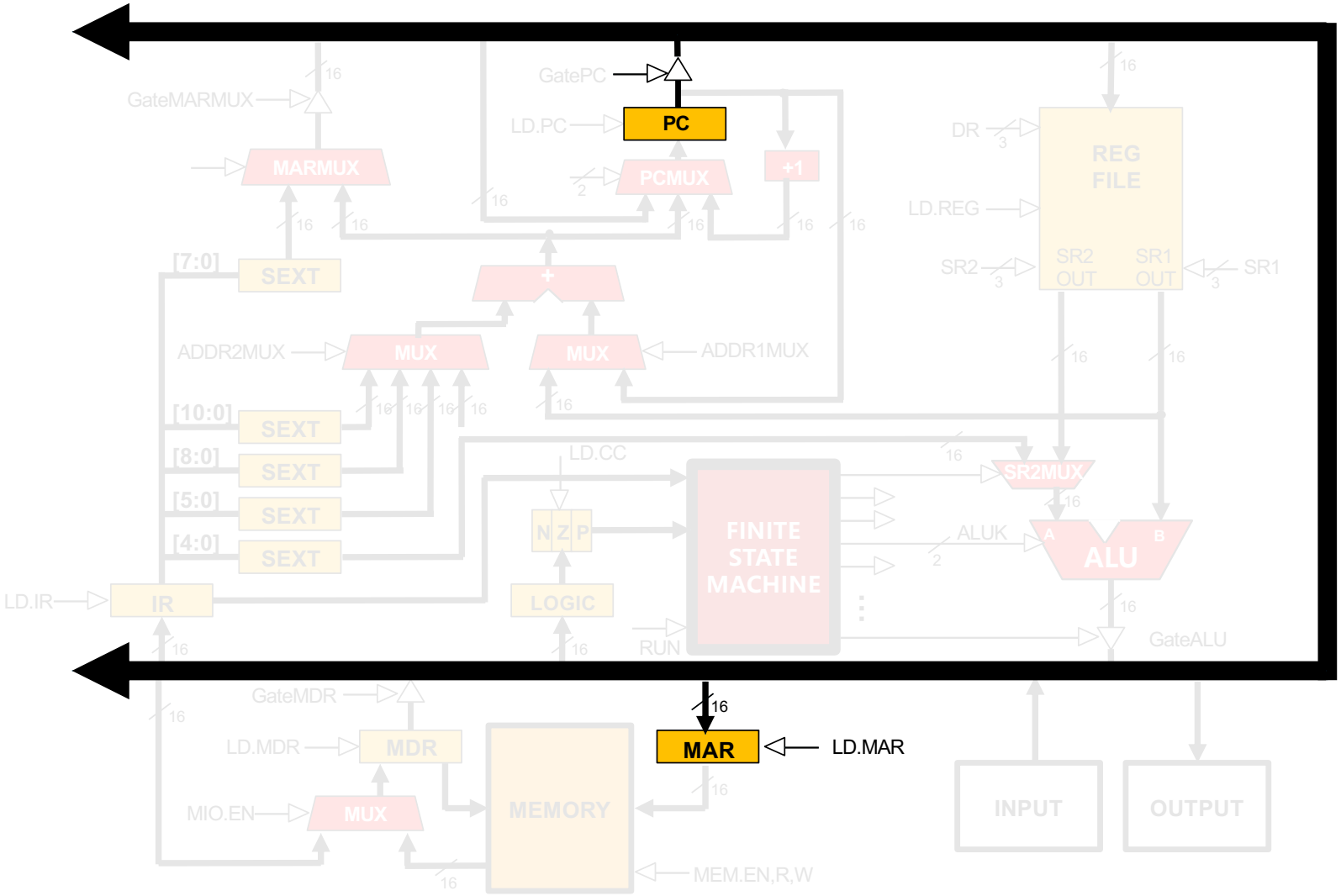


- Just like JMP (but PC is saved in R7)

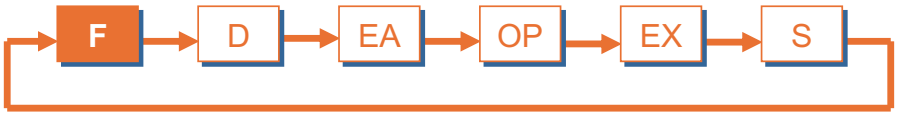
If the following JSR instruction is stored in location x4200,
its execution will cause the PC to be loaded with x3E05
(i.e., **xFC04 + x4201**) and R7 to be loaded with x4201

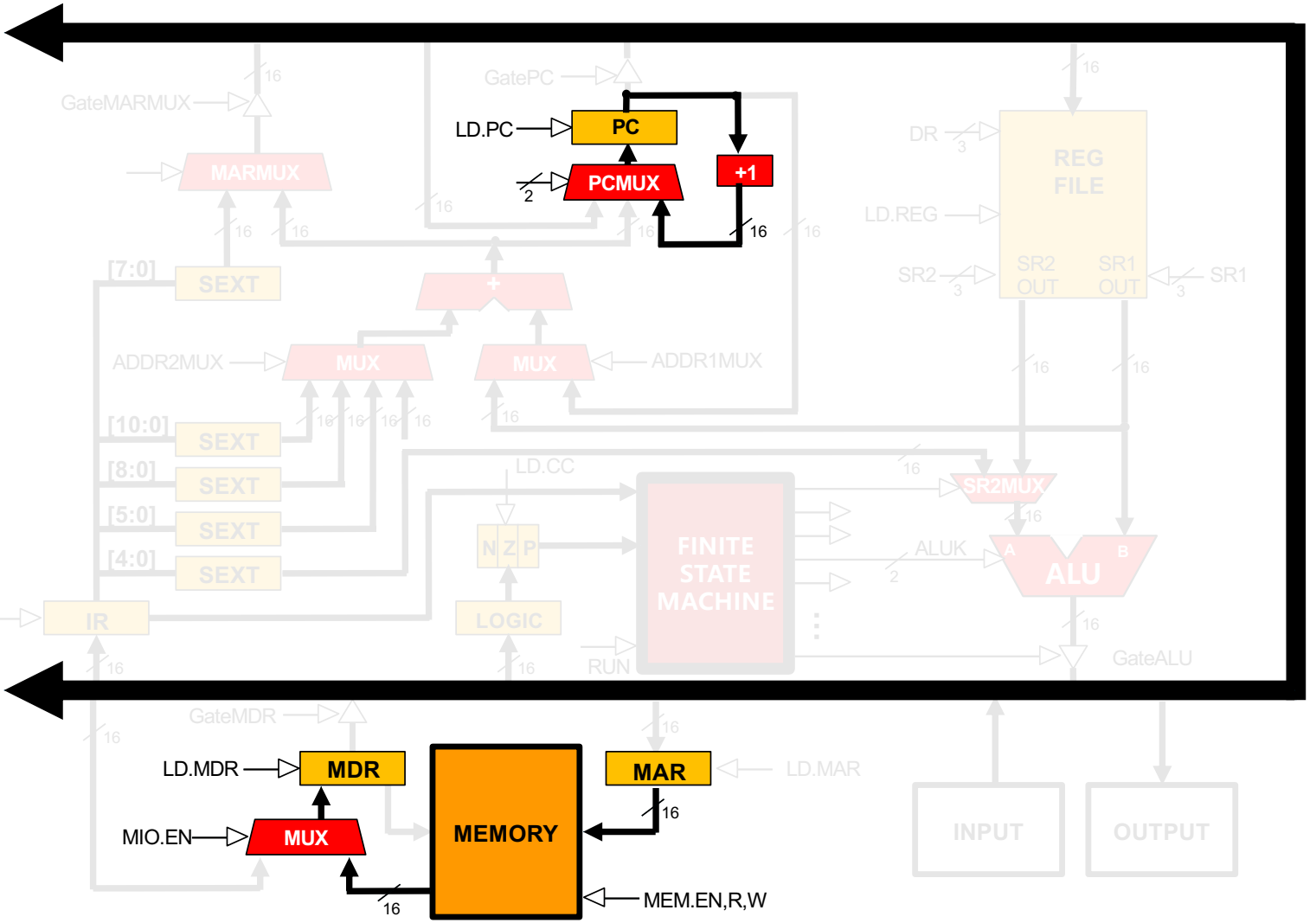
JSR (PC-Relative)



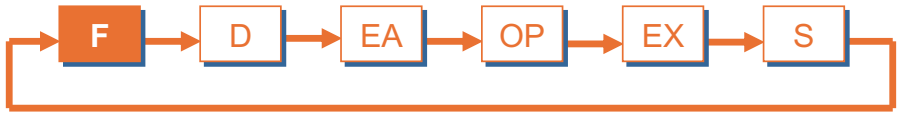


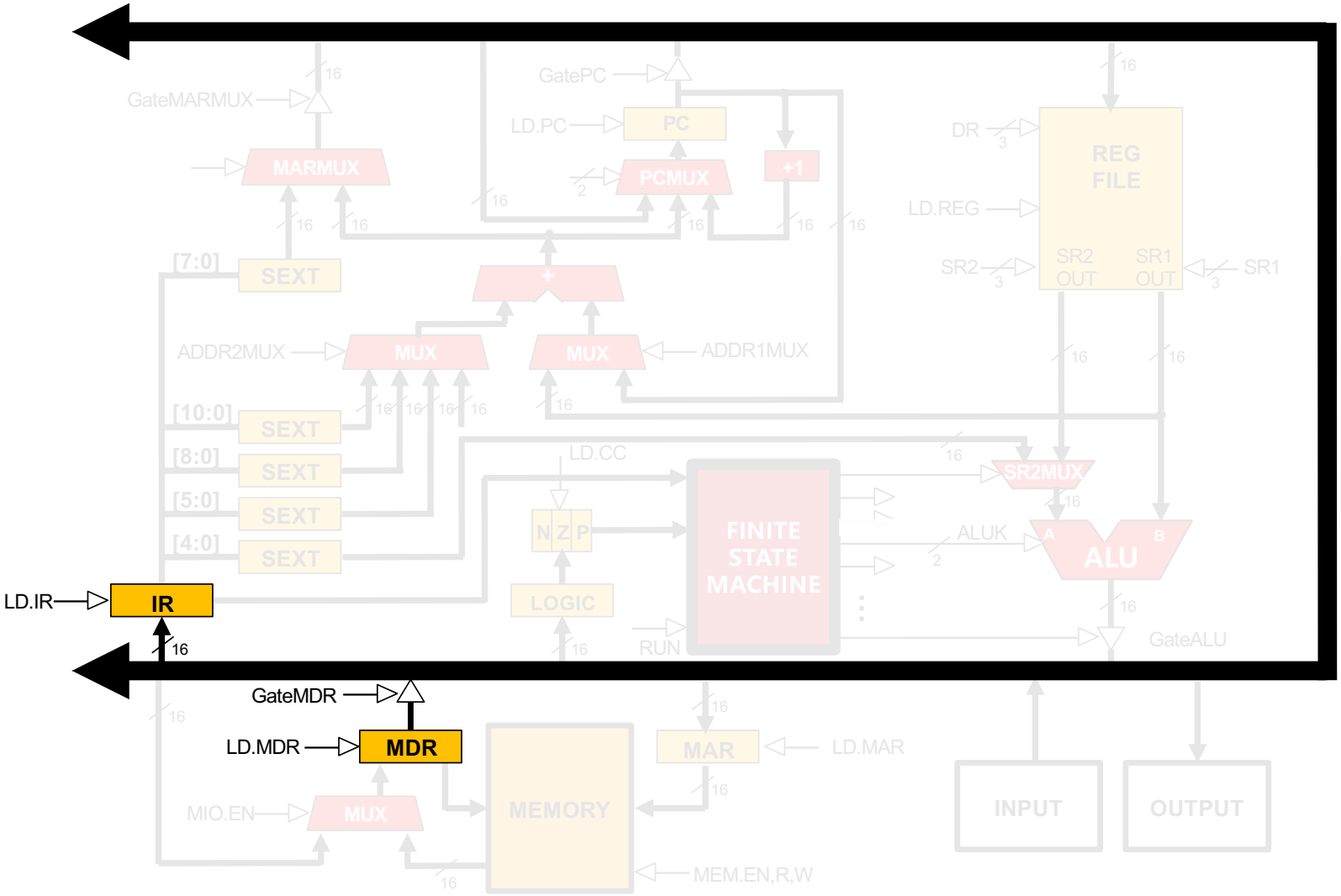
JSR (PC-Relative)



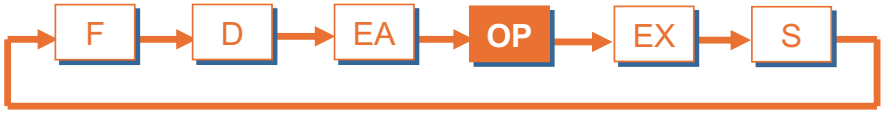


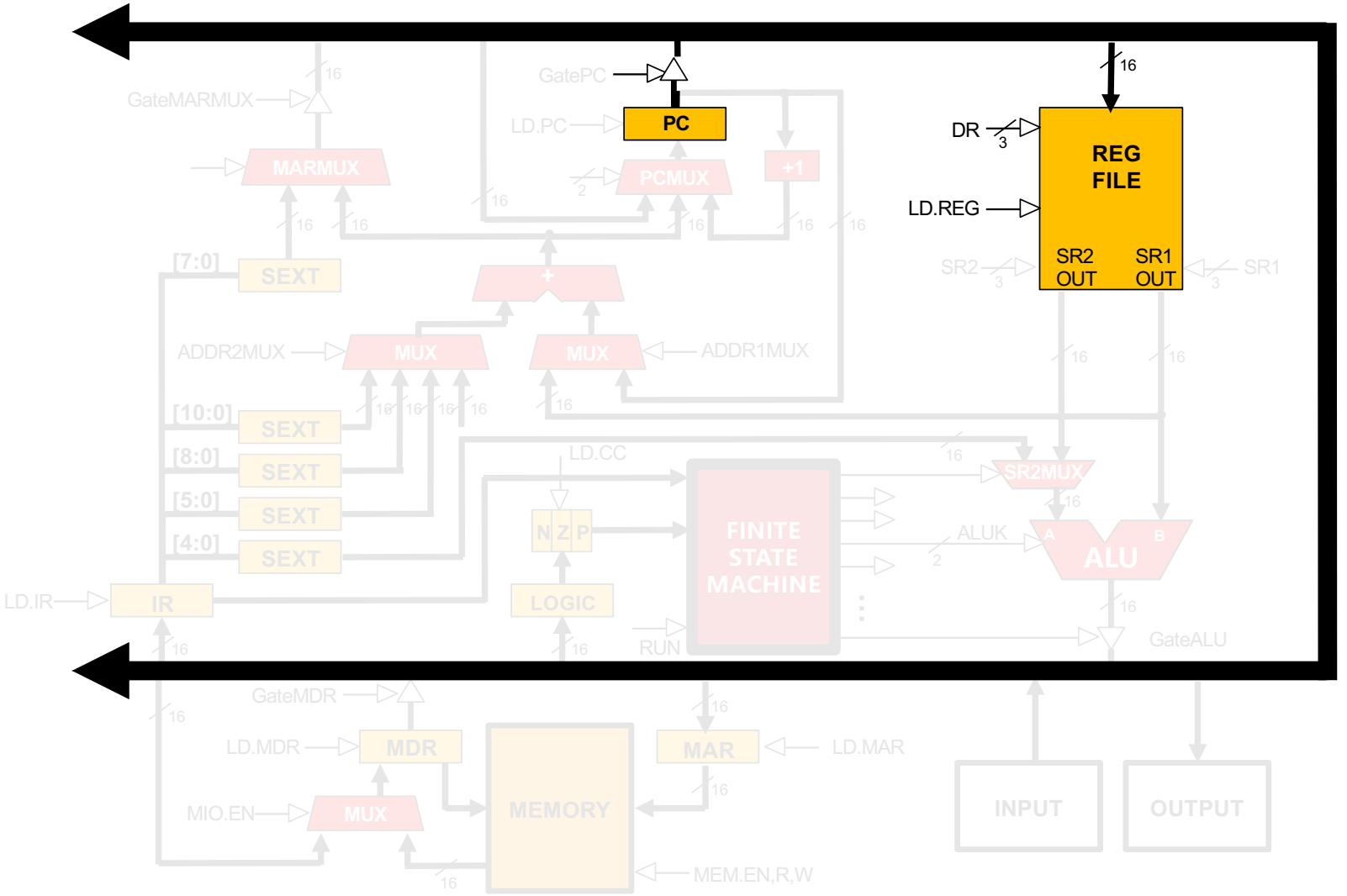
JSR (PC-Relative)



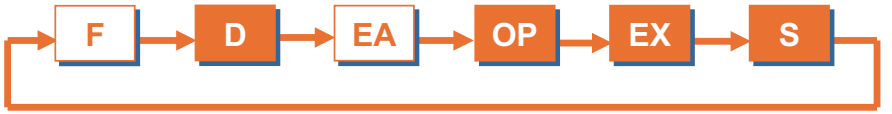


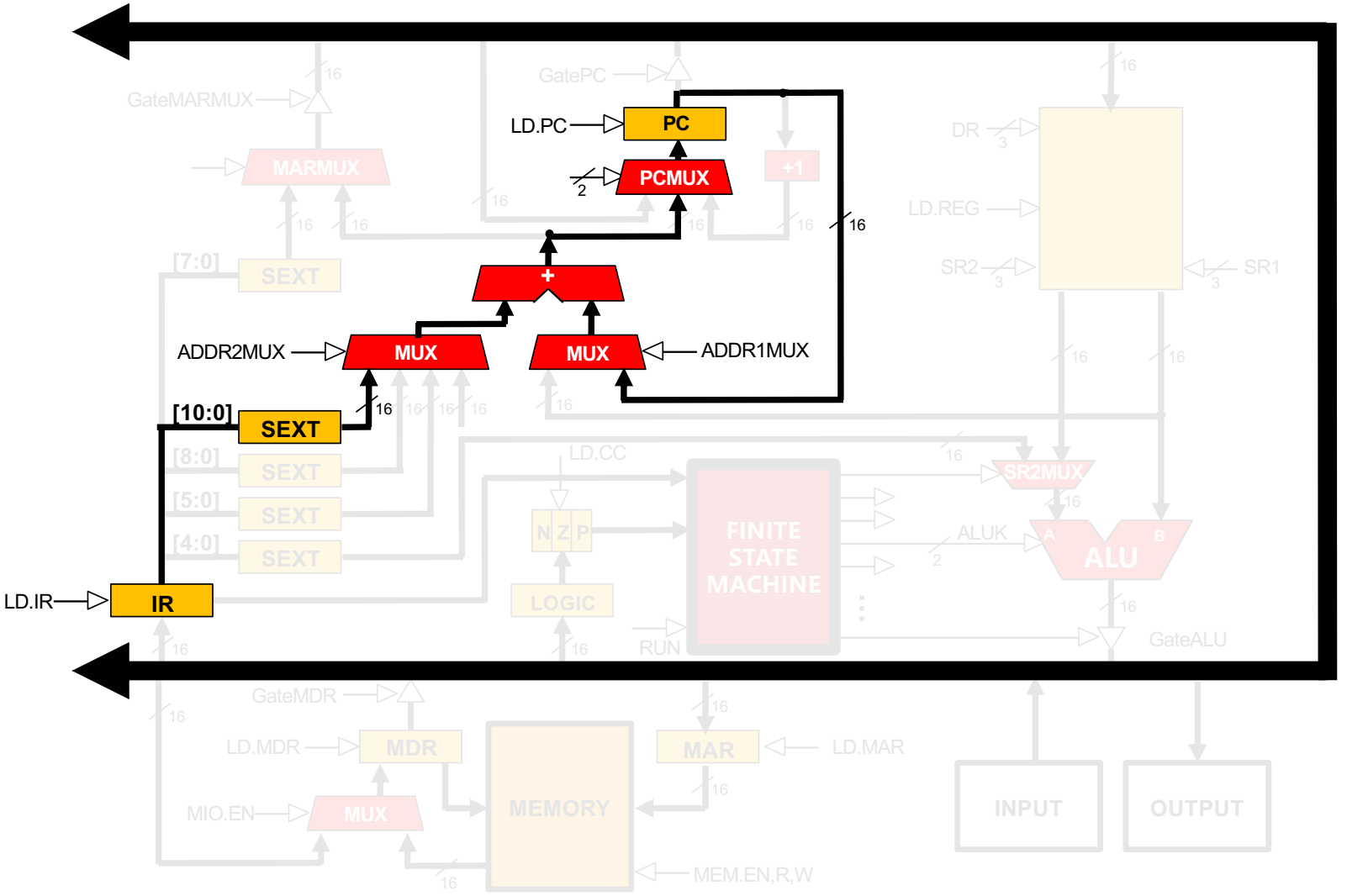
JSR (PC-Relative)





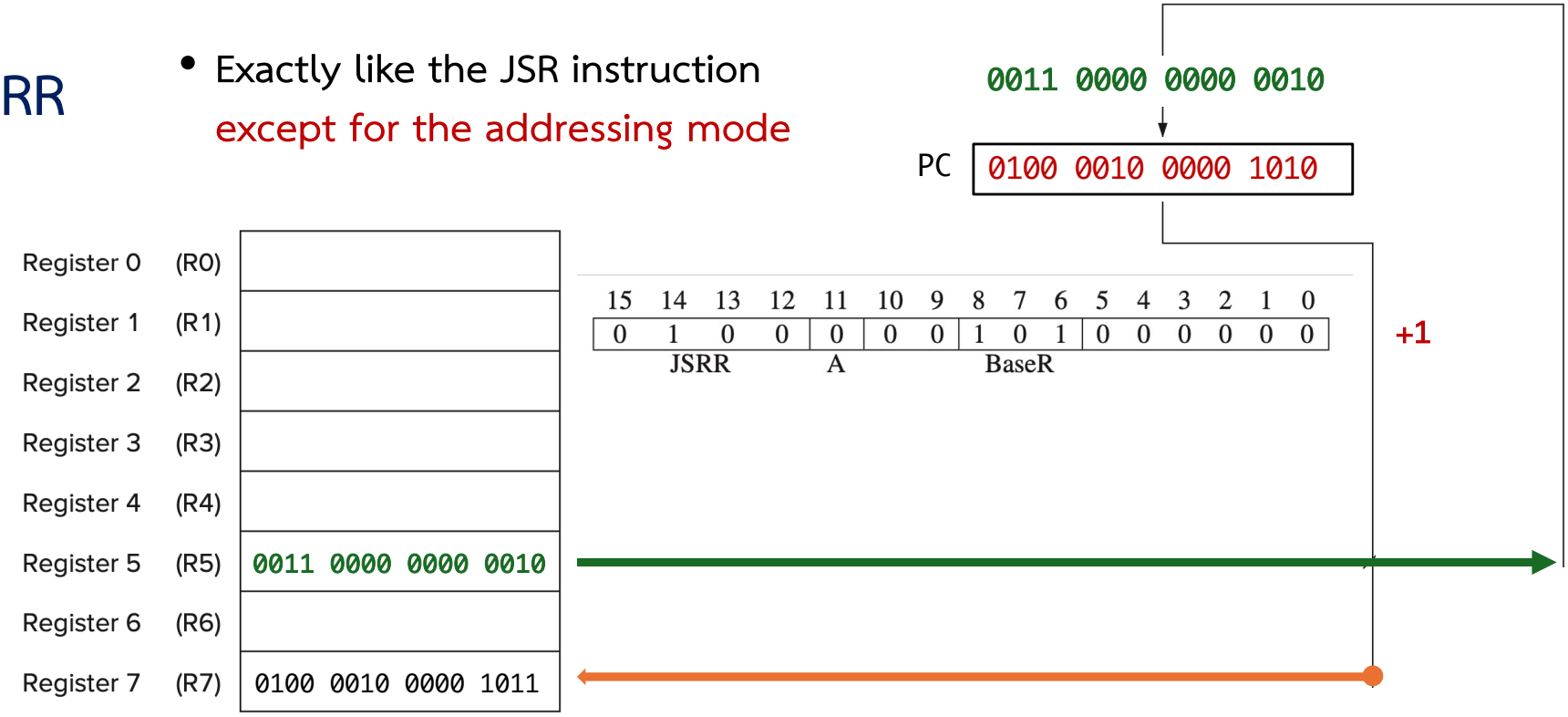
JSR (PC-Relative)





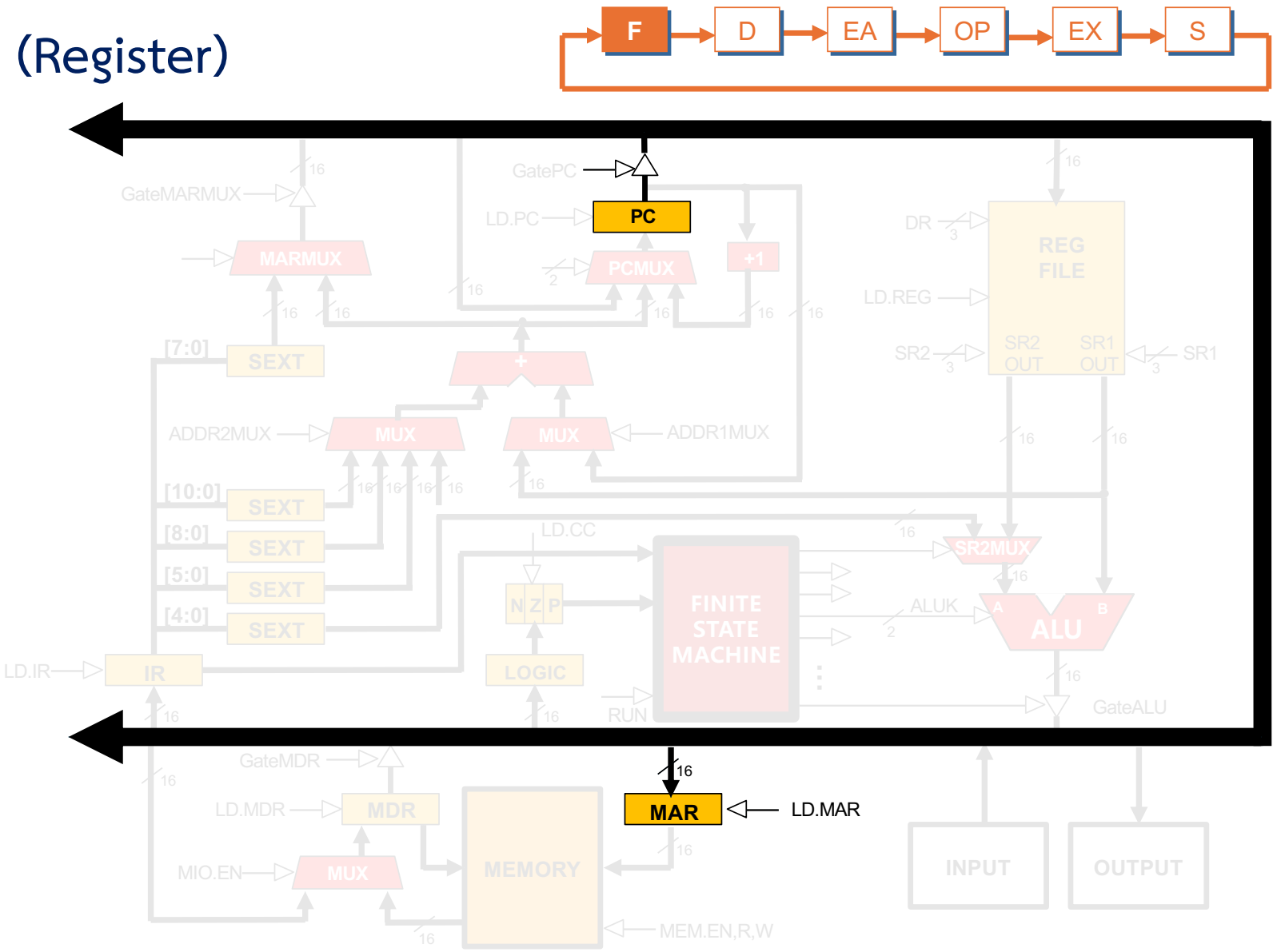
JSRR

- Exactly like the JSR instruction
  except for the addressing mode

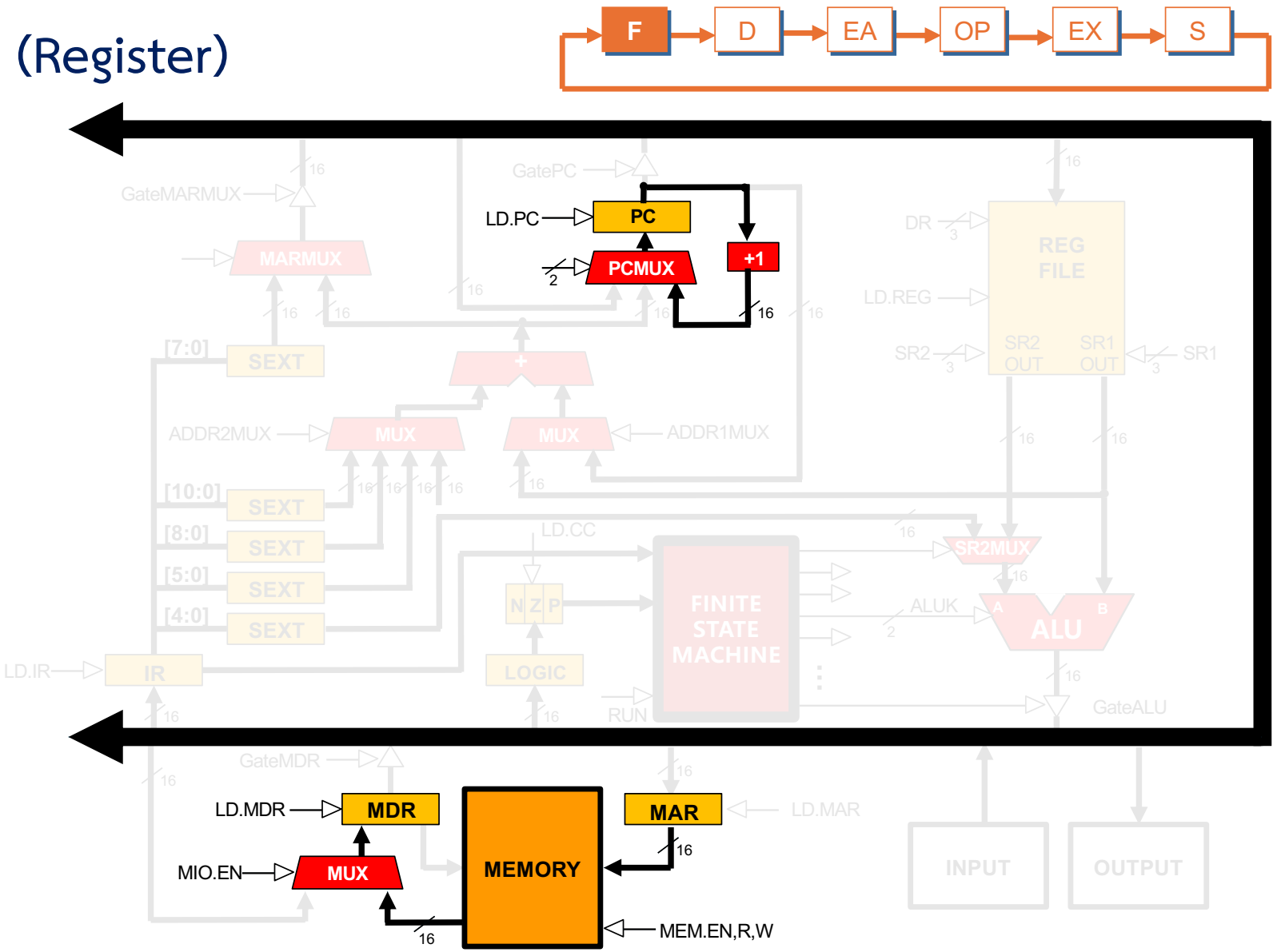


If the following JSRR instruction is stored in location x420A, and if R5 contains x3002, the execution of the JSRR will cause R7 to be loaded with x420B and the PC to be loaded with x3002

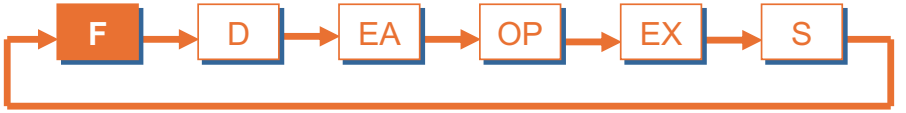
JSRR (Register)

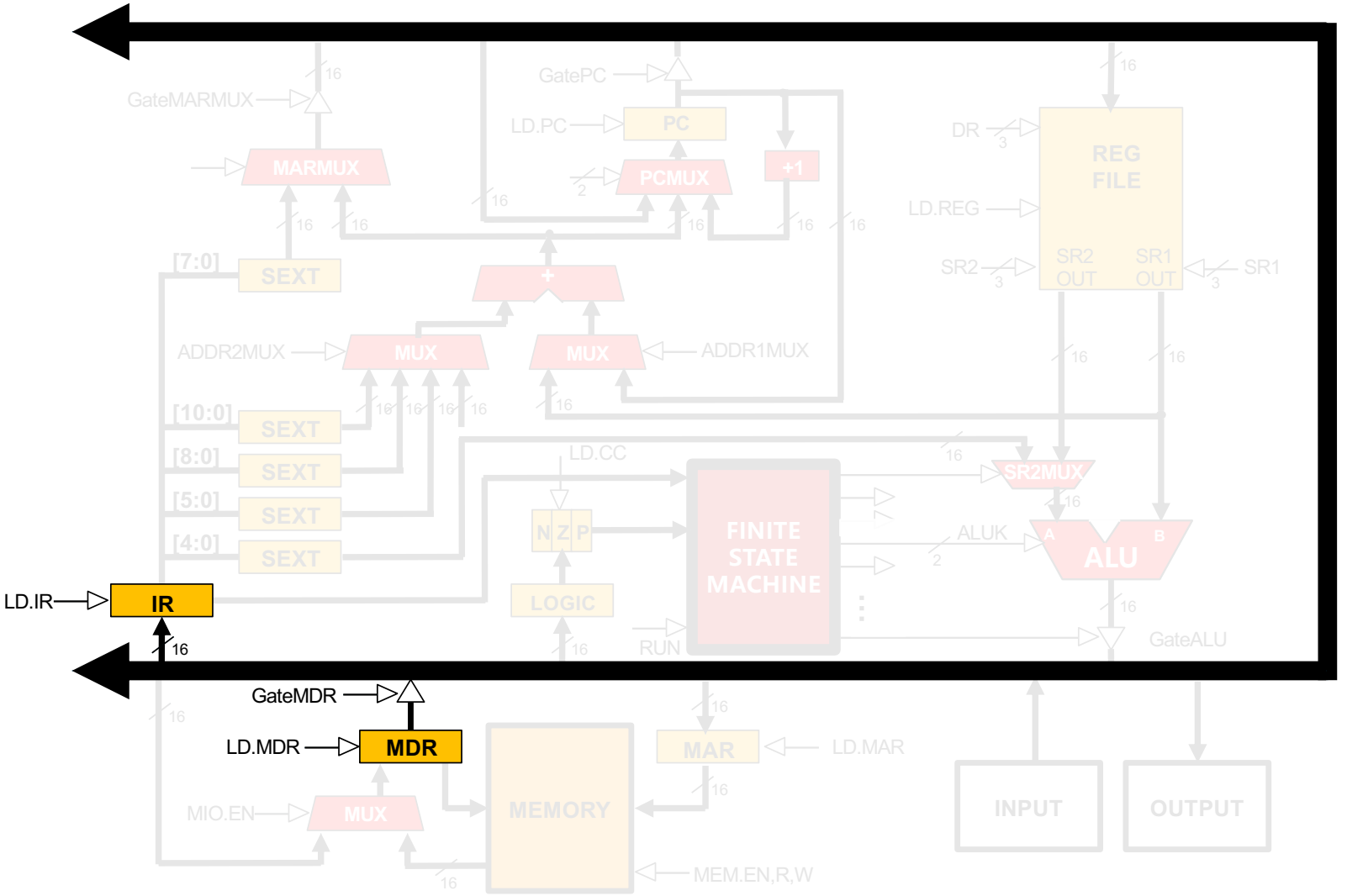


JSRR (Register)

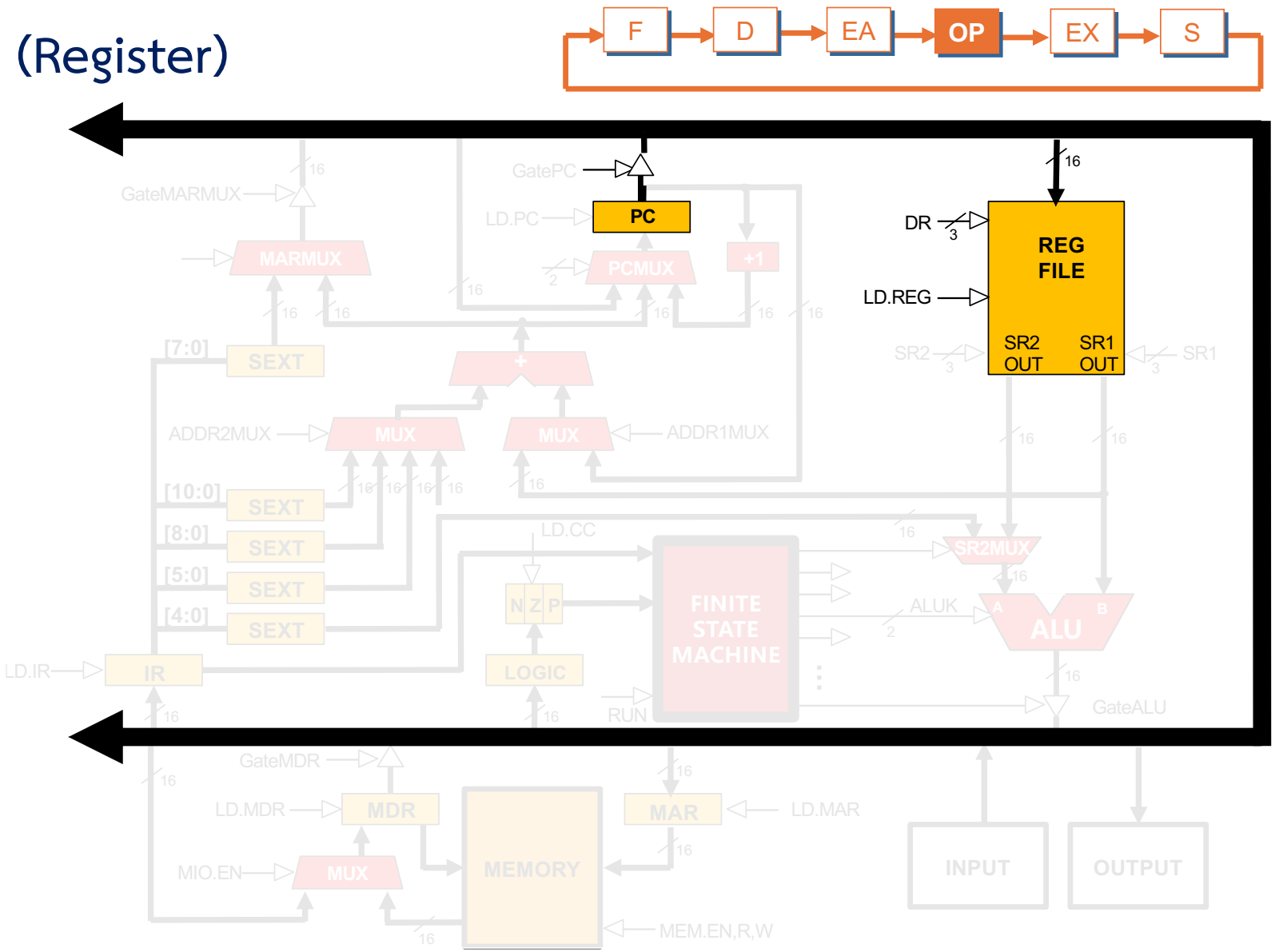


JSRR (Register)

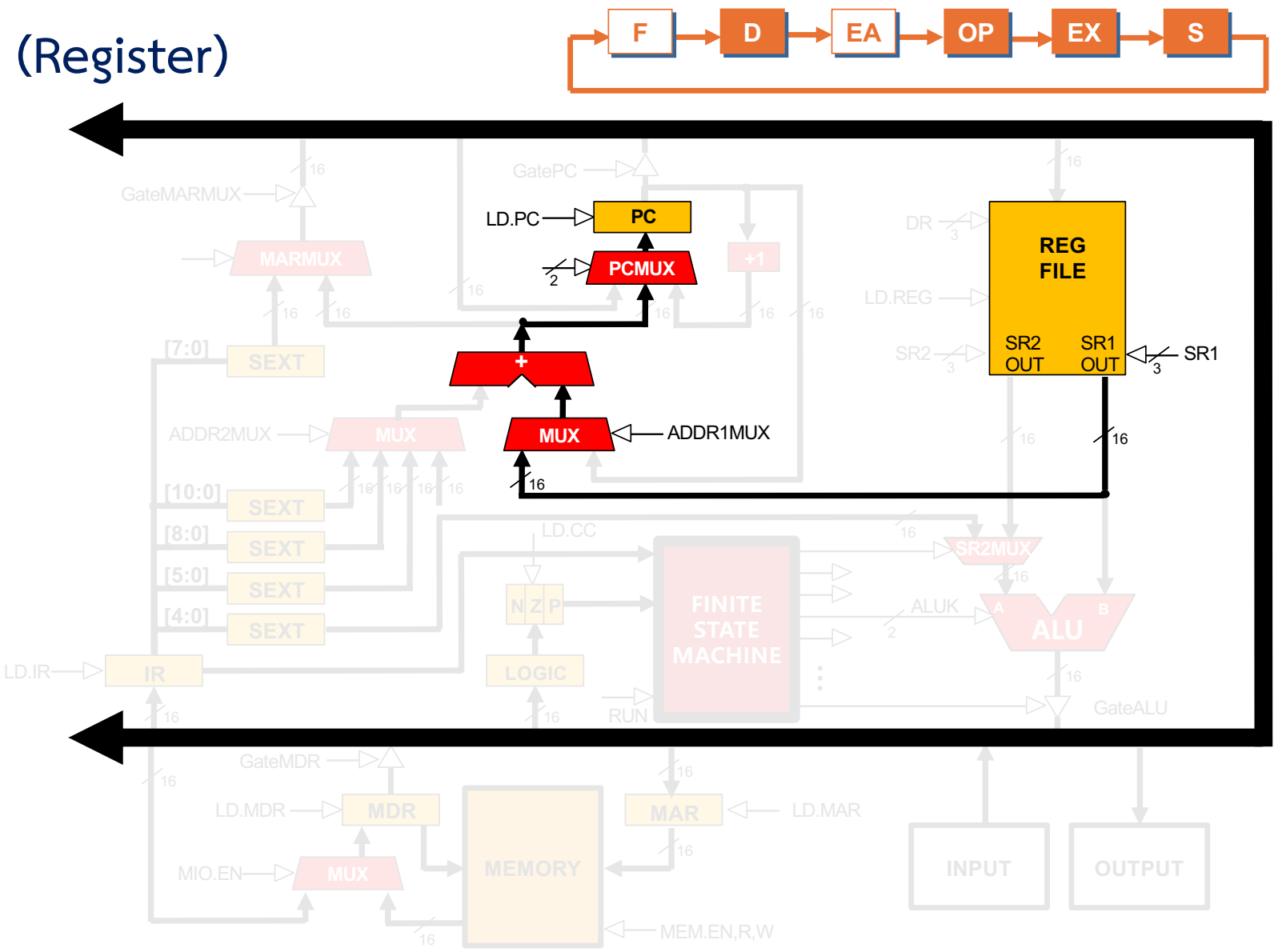




JSRR (Register)

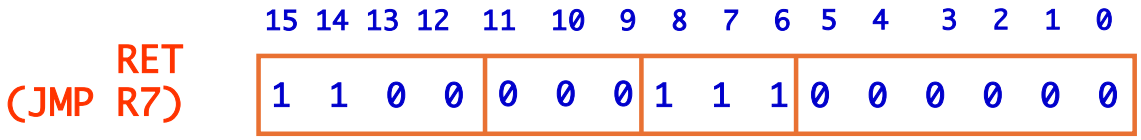


JSRR (Register)



RET instruction

- RET – return instruction
  - Place address in R7 in PC, Return the execution to the last calling point.
  - PC ← (R7)



Example: Negate the value in R0

```
TwosComp    NOT    R0, R0        ;flip bits
            ADD    R0, R0, #1    ;add one
            RET                  ;return to caller
```

To call from a program

```
;need to compute R4 = R1-R3
            ADD    R0, R3, #0    ;copy R3 to R0
            JSR    TwosComp      ;negate
            ADD    R4, R1, R0    ;add to R1
            ...
```

Using Subroutines

- Programmer must know
  - Address: or at least a label that will be bound to its address
  - Function: what it does
    - NOTE: The programmer does not need to know *how* the subroutine works, but what changes are visible in the machine's state after the routine has run
  - Arguments: what they are and where they are placed
  - Return values: what they are and where they are placed

Passing Information To Subroutines

- Argument(s)
  - Value passed into a subroutine is called an argument
  - This is a value needed by the subroutine to do its job
- How?
  - In registers (simple, fast, but limited number)
  - In memory (many, inconvenient, expensive)
  - Both

```
int sum (a, b)
    return a+b
```

Getting Values From Subroutines

- Return Values
  - A value passed out of a subroutine is called a return value
  - This is the value that you called the subroutine to compute

- How?
  - Registers, memory, or both
  - Single return value in register most common

```
int sum (a, b)
    return a+b
```

Saving and Restore Registers

- Like service routines, must save and restore registers
  - Who saves what is part of the calling convention
- Same as trap service routines
- Save anything that subroutine alters internally that shouldn't be visible when the subroutine returns
- Restore incoming arguments to original values (unless overwritten by return value)
- Remember
  - You MUST save R7 if you call any other subroutine or trap ***
  - Otherwise, you won't be able to return!

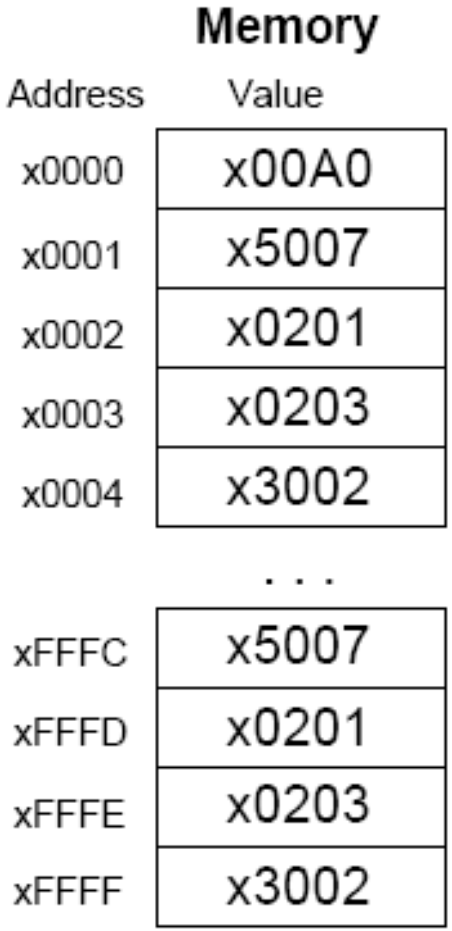
Subroutine Template

```
01 SUB_NAME
02       ;Register Saving
03       ST R0, SUB_R0
04       ST R1, SUB_R1
05       …
06       ST R6, SUB_R6
07       ST R7, SUB_R7;Return address
08
09       ;***Code***
10       ;……
11       ;Register Restoring
12       LD R0, SUB_R0
13       LD R1, SUB_R1
14       …
15       LD R6, SUB_R6
16       LD R7, SUB_R7          ;Return address
17       RET
```

Memory Model for Program Execution

Using Memory

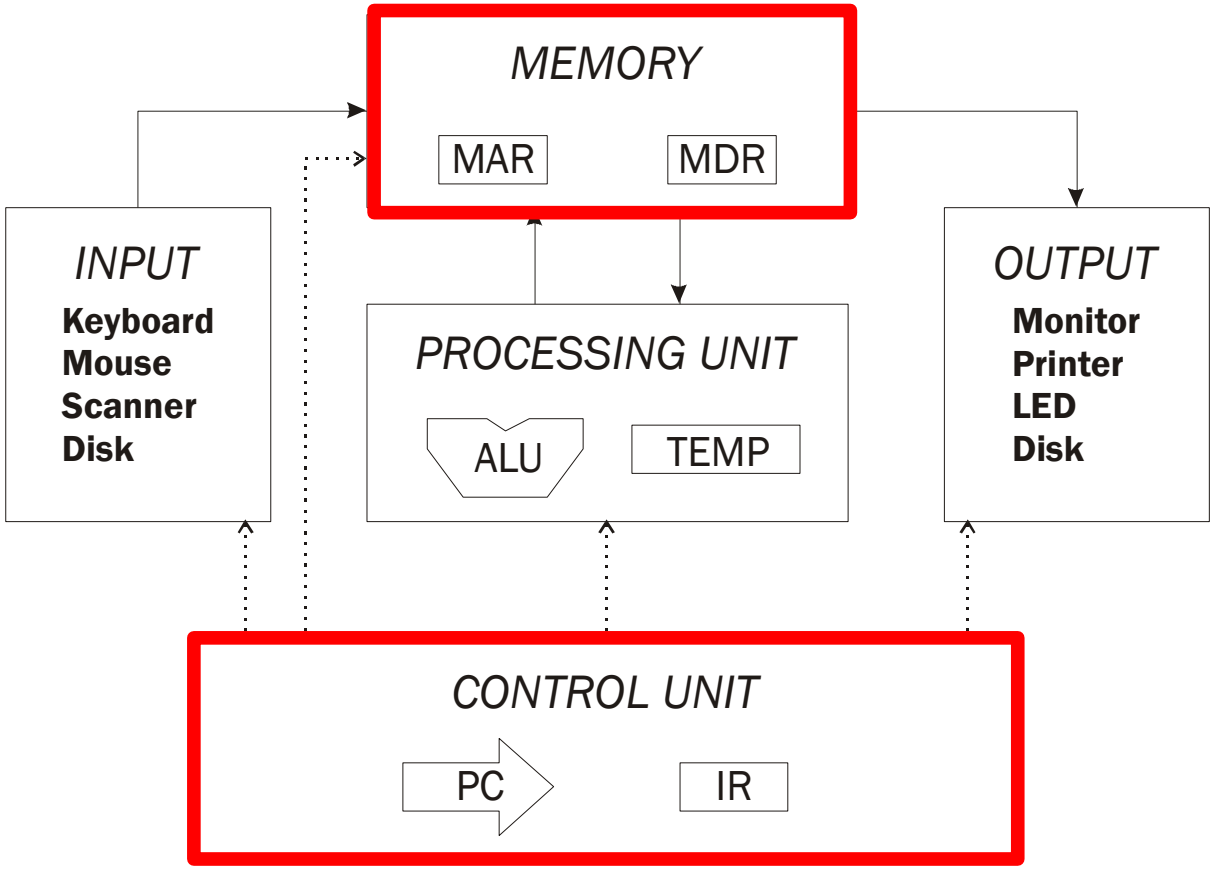
- Memory
  - Just a big “array”
  - “Indexed” by address
  - Accessed with loads and stores
- LD/LDR/LDI
  - Read a word out of memory
  - Use different addressing mode
- ST/STR/STI
  - Place a word in memory
  - Use different addressing mode



Using Memory

- Problem
  - What if the memory you want to access is far away?
  - LD/ST won't work (PC-relative)
  - LDR/STR won't work alone (need to get address in register)
- Solution: LDI/STI
  - Place *address* of far away value nearby
  - Load address, then load/store from that

Memory Model for Function Calls



Problem

- How do we allocate memory during the execution of a program written in C?
  - Programs need memory for code and data such as instructions, global and local variables, etc.
  - Modern programming practices encourage many (reusable) functions, callable from anywhere.
  - Some memory can be statically allocated, since the size and type is known at compile time.
  - Some memory must be allocated dynamically, size and type is unknown at compile time.

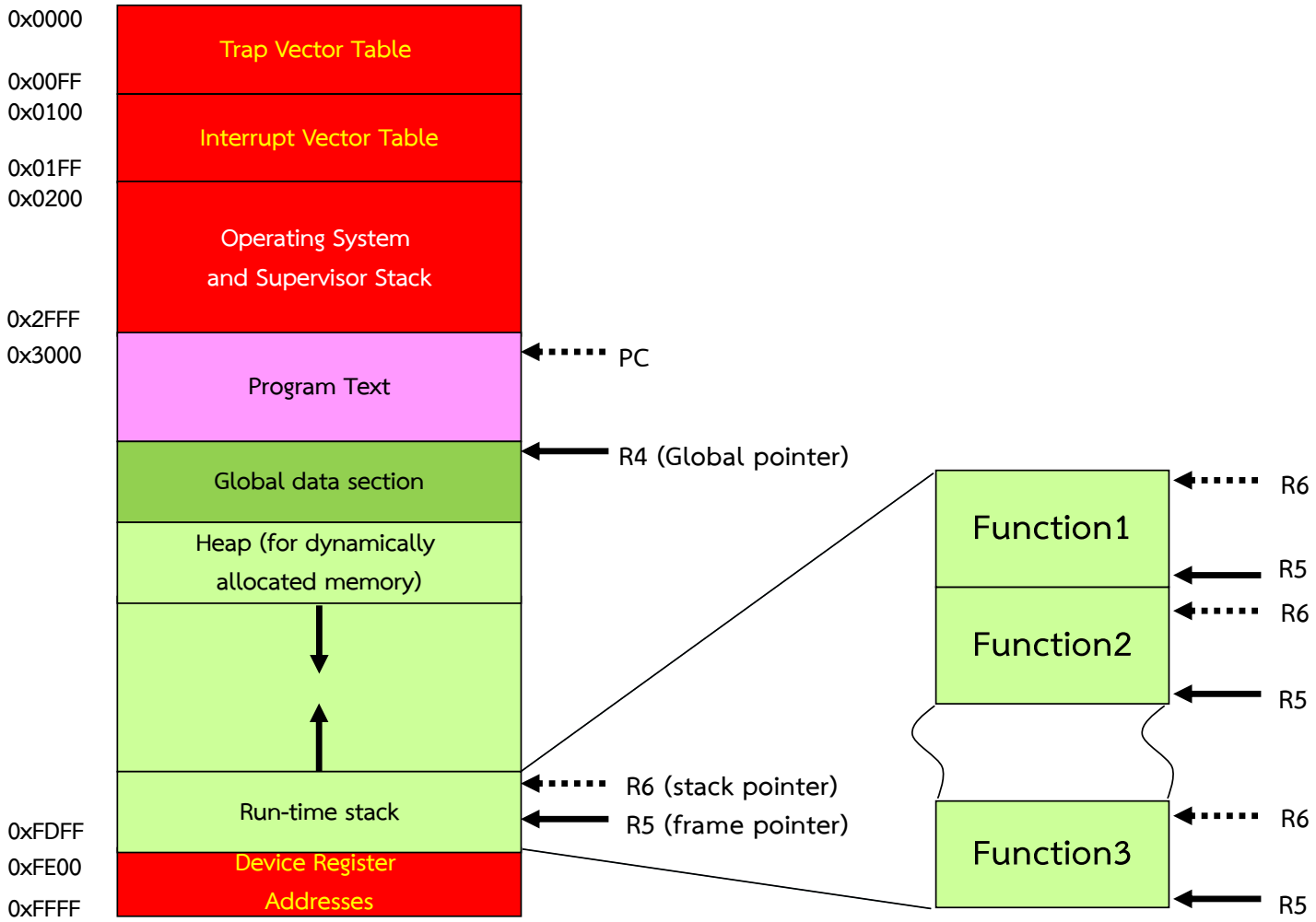
Motivation

- Why is memory allocation important?
- Why not just use a memory manager?
  - Allocation affects the performance and memory usage of every C, C++, Java program.
  - Current systems do not have enough registers to store everything that is required.
  - Memory management is too slow and heavy to solve the problem.
  - Static allocation of memory resources is too inflexible and inefficient, as we will see.

Goals

- What do we care about?
  - Fast program execution
  - Efficient memory usage
  - Avoid memory fragmentation
  - Maintain data locality
  - Allow recursive calls
  - Support parallel execution
  - Minimize resource allocation
  - Memory should never be allocated for functions that are not executed.

Memory Model in the LC-3



Stack


LIFO
LAST IN
LAST IN
FIRST OUT
FIRST OUT
FIRST OUT
FIRST OUT
Top
First Out

Abstract Data Types: Data Structures

- Up to now, we have processed a single value
  - an integer
  - an ASCII character
- The information in the real world is far more complex than simple, single numbers. We call these complex items of information abstract data types, or more colloquially data structures
  - a company's organization chart
  - a list of items arranged in alphabetical order
- We will study three abstract data types:
  - stacks
  - queues
  - and character strings

Stack: An Abstract Data Type

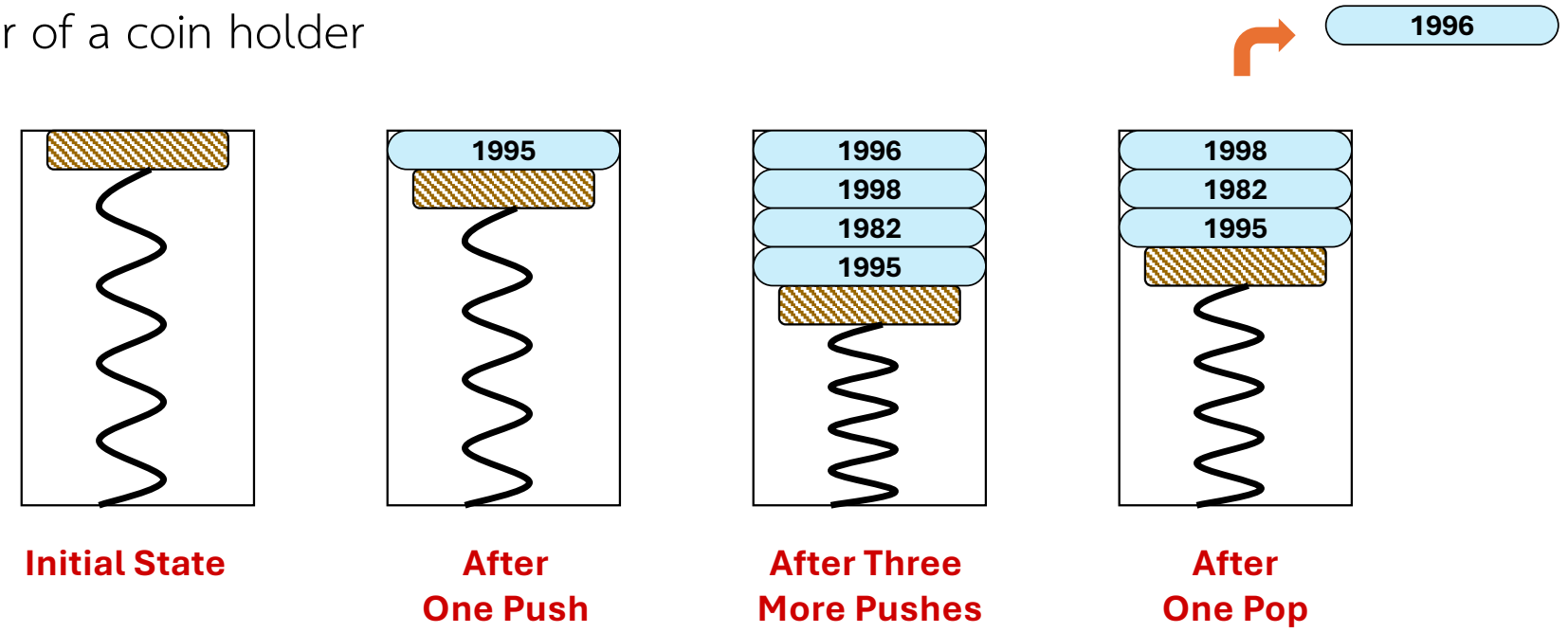
- An important abstraction that you will encounter in many applications.
- The fundamental model for execution of C, Java, Fortran, and many other languages.
- We will describe two uses of the stack:
  - Evaluating arithmetic expressions
    - Store intermediate results on stack instead of in registers
  - Function calls
    - Store parameters, return values, return address, dynamic link(*pointer to caller's stack frame*)
    - Interrupt-Driven I/O (*chapter 9*)
    - Store *processor state* for currently executing program

Stack Data Structure

- A LIFO (last-in first-out) storage structure
  - The first thing you put in is the last thing you take out
  - The last thing you put in is the first thing you take out
  - This means of access is what defines a stack, not the specific implementation
- Two main operations
  - PUSH: add an item to the stack
  - POP: remove an item from the stack
- Error conditions:
  - Underflow (try to pop from empty stack)
  - Overflow (try to push onto full stack)
- A register (eg. R6) holds address of top of stack (TOS)

A Physical Stack

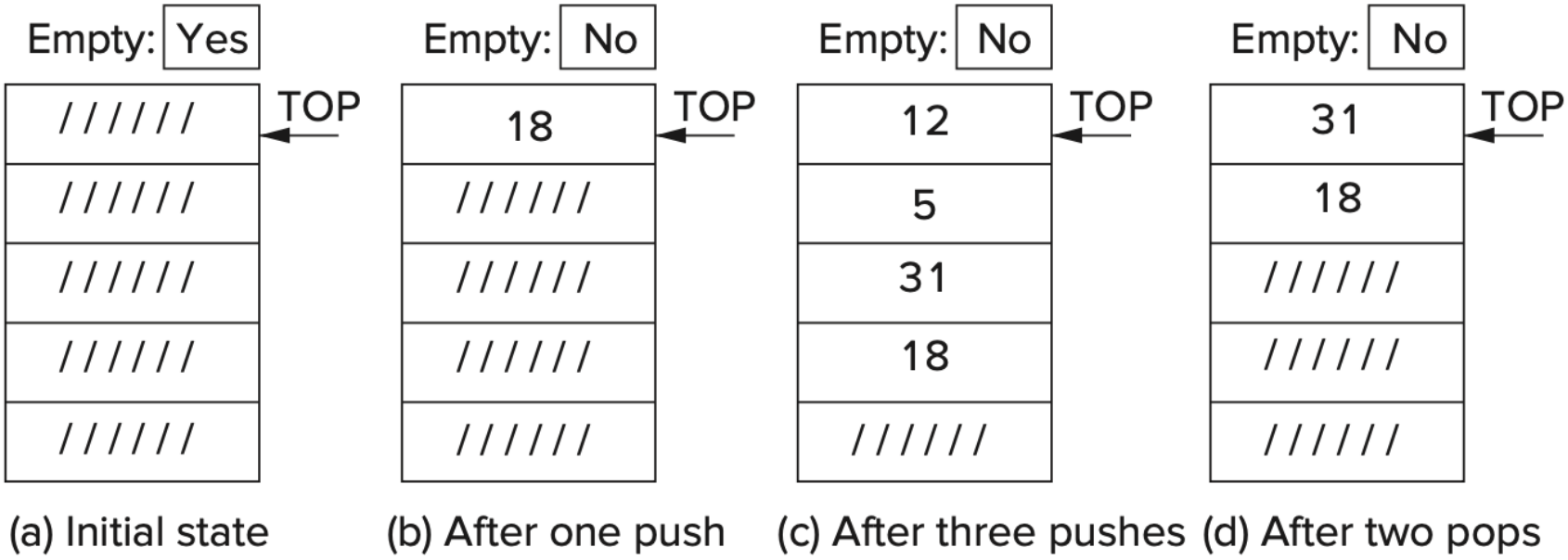
- Behavior of a coin holder



Last In is the First Out (LIFO)

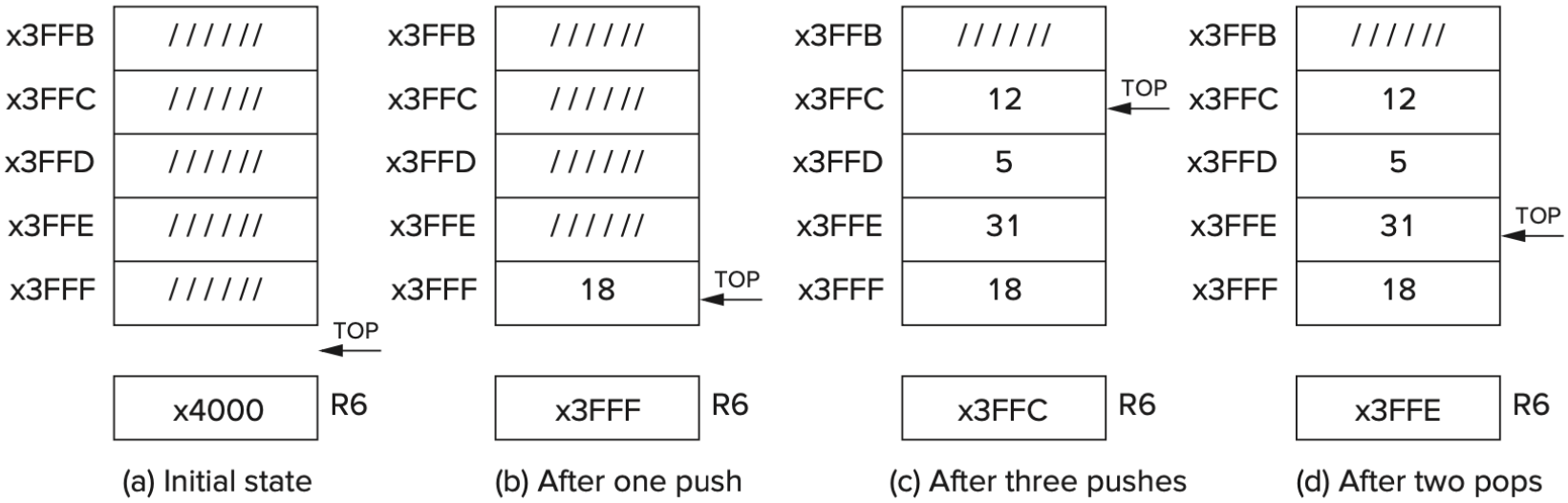
A Hardware Stack Implementation

- Data items move between registers
  - There are 5 registers in each of the following figures



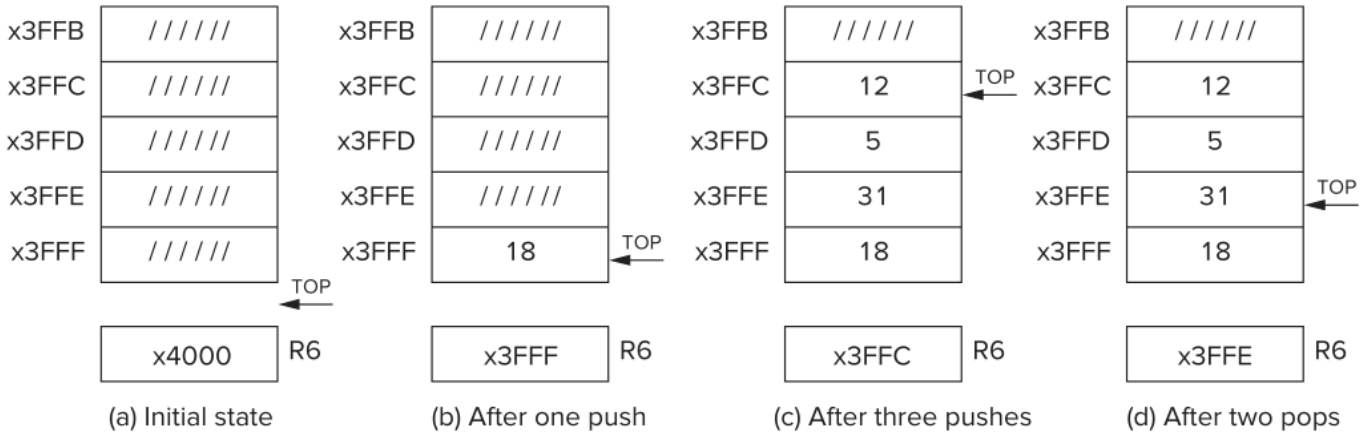
A Software Stack Implementation

- Data items don't move in memory, just our idea about where TOP of the stack is



- By convention, R6 holds the Top of Stack (TOS) Pointer (SP)

Basic Push and Pop Code




```
PUSH
        ADD R6, R6, #-1          ; increment stack ptr
        STR R0, R6, #0           ; store data(R0) to TOS
POP
        LDR R0, R6, #0           ; load data(R0) from TOS
        ADD R6, R6, #1           ; decrement stack ptr
```

Stacks can grow in either direction (toward higher address or toward lower addresses)

Pop with Underflow Detection

- If we try to pop too many items off the stack,

  an underflow condition occurs.

  - Check for underflow by checking TOS before removing data.
  - Return status code in R5 (0 for success, 1 for underflow)

```
POP    LD   R1, EMPTY
       ADD  R2, R6, R1   ; Compare stack pointer
       BRz  UNDER        ; with x4000
;
       LDR R0, R6, #0    ; The actual 'pop'
       ADD R6, R6, #1    ; Adjust stack pointer
;
       AND R5, R5, #0    ; Success: return R5 = 0
       RET
UNDER  AND R5, R5, #0    ; Underflow: return R5 = 1
       ADD R5, R5, #1
       RET
EMPTY  .FILL xC000              ; EMPTY = -x4000
```

Push with Overflow Detection

- If we try to push too many items onto the stack,

  an overflow condition occurs.

  - Check for underflow by checking TOS before adding data.
  - Return status code in R5 (0 for success, 1 for overflow)

```
PUSH   LD   R1, MAX
       ADD  R2, R6, R1        ; Compare stack pointer
       BRz  OVER              ; with MAX
       ADD  R6, R6, #-1       ; Adjust stack pointer
       STR  R0, R6, #0        ; The actual 'push'
       AND  R5, R5, #0        ; Success: return R5 = 0
       RET
OVER   AND  R5, R5, #0
       ADD  R5, R5, #1        ; Overflow: return R5 = 1
       RET
MAX    .FILL xC005            ; MAX = -x3FFB
```

PUSH & POP in LC-3 - 1

```
01 ;
02 ; Subroutines for carrying out the PUSH and POP functions. This
03 ; program works with a stack consisting of memory locations x3FFF
04 ; through x3FFB. R6 is the stack pointer .
05 ;
06 POP   AND R5,R5,#0      ; R5 <-- success
07       ST R1,Save1       ; Save registers that
08       ST R2,Save2       ; are needed by POP
09       LD R1,EMPTY       ; EMPTY contains -x4000
0A       ;
0B       ADD R2,R6,R1      ; Compare stack pointer to x4000
0C       BRz fail_exit     ; Branch if stack is empty
0D       ;
0E       LDR R0,R6,#0      ; The actual "pop"
0F       ADD R6,R6,#1      ; Adjust stack pointer
10       BRnzp success_exit
11 ;
12 PUSH  AND R5,R5,#0
13       ST R1,Save1       ; Save registers that
14       ST R2,Save2       ; are needed by PUSH
15       LD R1,FULL        ; FULL contains -x3FFB
16       ADD R2,R6,R1      ; Compare stack pointer to x3FFB
17       BRz fail_exit     ; Branch if stack is full
18 ;
```

PUSH & POP in LC-3 - 2

```
19                ADD R6,R6,#-1   ; Adjust stack pointer
1A                STR R0,R6,#0    ; The actual "push"
1B success_exit
                  LD R2,Save2 ; Restore original
1C                LD R1,Save1 ; register values
1D                RET
1E ;
1F fail_exit      LD R2,Save2 ; Restore original
20                LD R1,Save1 ; register values
21                ADD R5,R5,#1 ; R5 <-- failure
22                RET
23 ;
24 EMPTY .FILL xC000 ; EMPTY contains -x4000
25 FULL .FILL xC005 ; FULL contains -x3FFB
26 Save1 .FILL x0000
27 Save2 .FILL x0000
```

Arithmetic Using a Stack (p387, chapter 10.2)

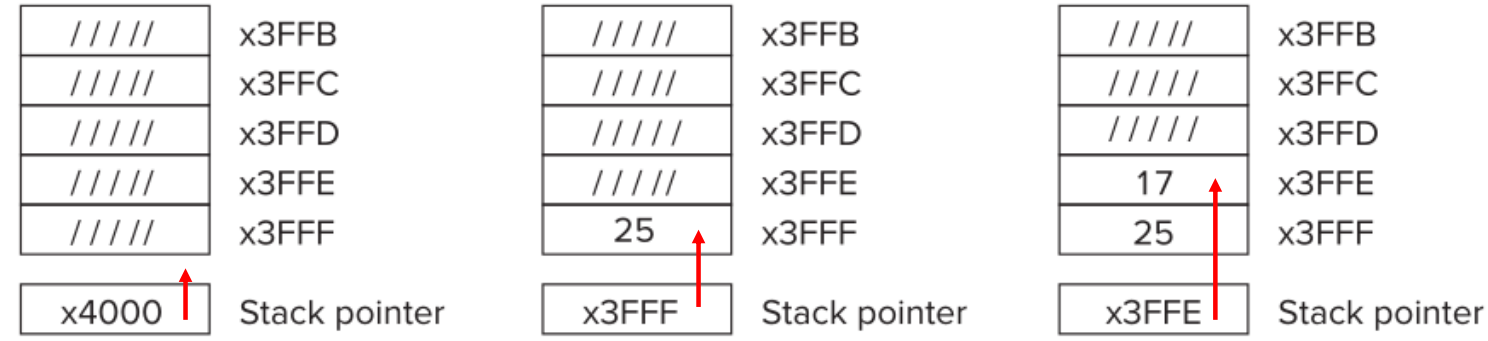
- Instead of registers, some ISA's use a stack for source and destination operations.
- The computer always pops and pushes operands from the stack, and hence no addresses need to be specified explicitly. Therefore, stack machines are sometimes referred to as zero-address machines.
  - Example: ADD instruction pops two numbers from the stack, adds them, and pushes the result to the stack.

ADD vs. ADD R0, R1, R2

- Evaluating (A+B)·(C+D) using a stack:

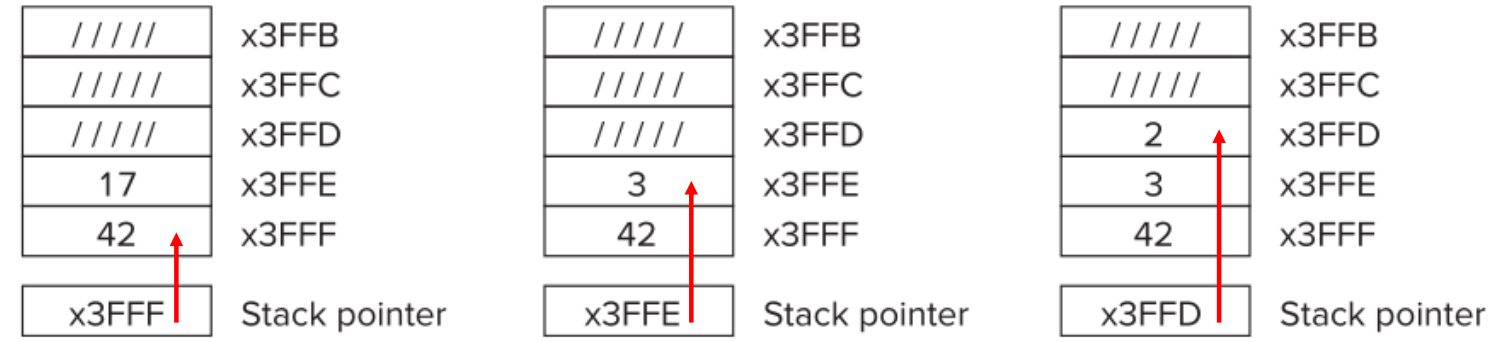
```
push A
push B
ADD
push C
push D
ADD
MULTIPLY
pop result
```

(25+17) x (3+2)



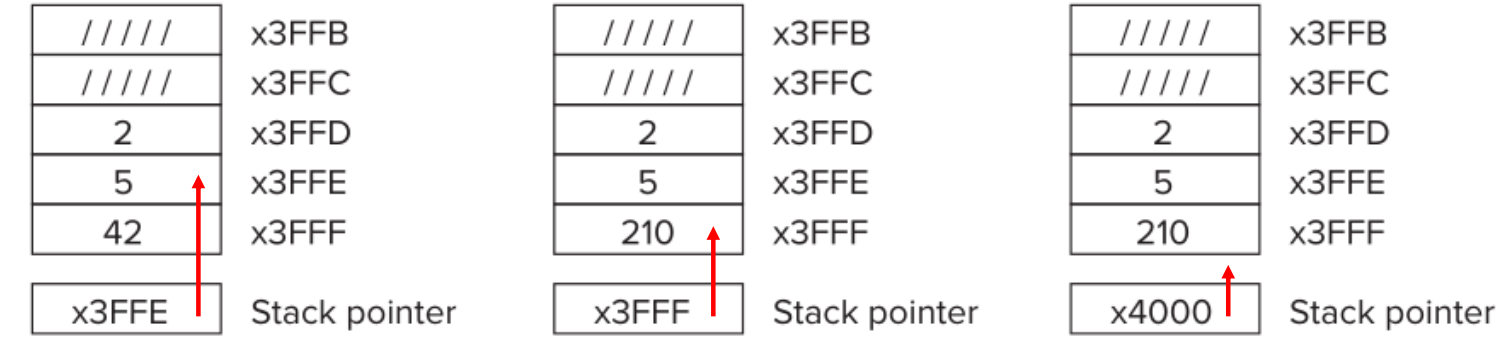
(a) Before (b) After first push (c) After second push

(25+17) x (3+2)



(d) After first add (e) After third push (f) After fourth push

(25+17) x (3+2)



(g) After second add (h) After multiply (i) After pop

Implementing Functions in C

Using a Stack

Function in C

- Smaller, simpler, subcomponent of program
- Provides abstraction
  - hide low-level details
  - give high-level structure to programmer, easier to understand overall program flow
  - enables separable, independent development
- C functions
  - zero or multiple arguments passed in
  - single result returned (optional)
  - return value is always a particular type
- In other languages, called procedures, subroutines, ...

Example of High-Level Structure

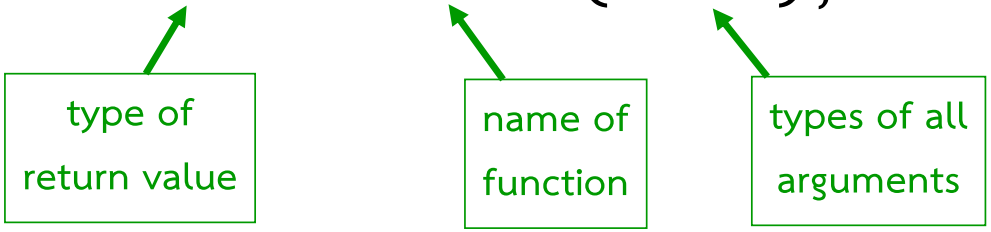
```
main()
{
  SetupBoard();  /* place pieces on board */
  DetermineSides();  /* choose black/white */

  /* Play game */
    do {
      WhitesTurn();
      BlacksTurn();
    } while (NoOutcomeYet());
}
```

Functions in C

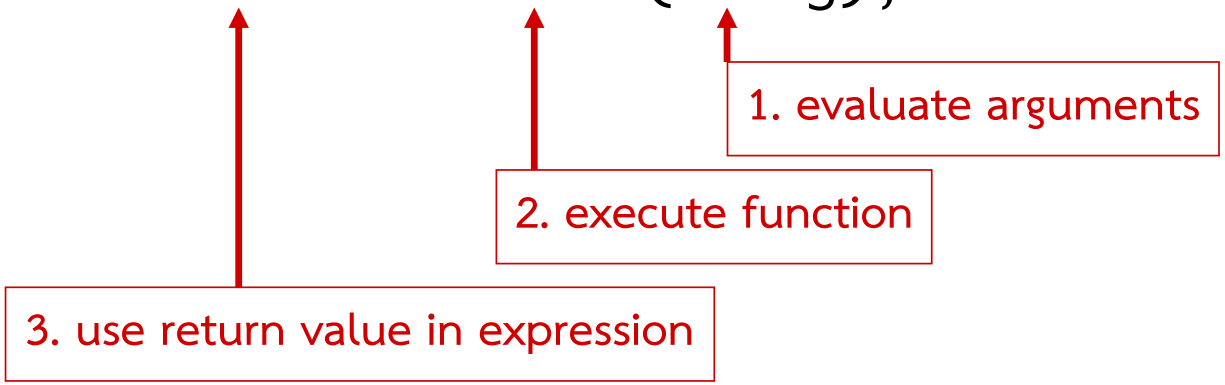
- Declaration (also called prototype)

```
int Factorial(int n);
```



- Function call -- used in expression

```
a = x + Factorial(f + g);
```



Function Definition

- State type, name, types of arguments
  - must match function declaration
  - give name to each argument (doesn't have to match declaration)

```
int Factorial(int n)
{
  int i;
  int result = 1;
  for (i = 1; i <= n; i++)
    result *= i;
  return result;
}
```

gives control back to calling function and returns value

Why Declaration?

- Since function definition also includes return and argument types, why is declaration needed?
- Use might be seen before definition.
  - Compiler needs to know return and arg types and number of arguments.
- Definition might be in a different file, written by a different programmer.
  - include a "header" file with function declarations only
  - compile separately, link together to make executable

Example

```
double ValueInDollars(double amount, double rate);

main()
{
  ...
  dollars = ValueInDollars(francs,
                           DOLLARS_PER_FRANC);
  printf("%f francs equals %f dollars.\n",
         francs, dollars);
  ...
}

double ValueInDollars(double amount, double rate)
{
  return amount * rate;
}
```

declaration

function call (invocation)

definition

Storage Requirements

- Code must be stored in memory so that we can execute the function
- Parameters must be sent from the caller to the callee so that the function receives them
- Local variables for the function must be stored somewhere, is one copy enough?
- Return address must be stored so that control can be returned to the caller
- Return values must be sent from the callee to the caller, that's how results are returned

Function Call in C

- Consider the following code:

```
// main program
Int a = 10;
Int b = 20;
Int c = foo(a,b);
Int foo(int x,int y)
{
   Int z;
   z= x+y;
   return z;
}
```

- What needs to be stored?
  - Code, parameters, local/global variables, return address/values

Possible Solution: Mixed Code and Data

- Function implementation:

```
foo             BR  foo_begin  ;skip over data
foo_rv          .BLKW 1                ;return value
foo_ra          .BLKW 1                ;return address
foo_paramx      .BLKW 1                ;‘x’ parameter
foo_paramy      .BLKW 1                ;‘y’ parameter
foo_localz      .BLKW 1                ;‘z’ local
foo_begin       ST R7, foo_ra          ;save return
                …
                LD R7, foo_ra          ;restore return
                RET
```

- Can construct data section by appending foo_

Corresponding to the option 1 in textbook p.497

Possible Solution: Mixed Code and Data

- Calling sequence

```
ST R1, foo_paramx          ; R1 has ‘x’
ST R2, foo_paramy          ; R2 has ‘y’
JSR foo                    ; Function call
LD R3, foo_rv              ; R3 = return value
```

- Code generation is relatively simple.
- Few instructions are spent on moving data.

Possible Solution: Mixed Code and Data

- Advantages:
  - Code and data are close together
  - Conceptually easy to understand
  - Minimizes register usage for variables
  - Data persists through life of program
- Disadvantages:
  - Cannot handle recursion or parallel execution
  - Code is vulnerable to self-modification (Code must be marked as "writable")
  - Consumes resource for inactive functions

Possible Solution: Separate Code and Data

- Memory allocation

```
foo_rv          .BLKW 1         ; foo return value
foo_ra          .BLKW 1         ; foo return address
foo_paramx      .BLKW 1         ; foo ‘x’ parameter
foo_paramy      .BLKW 1         ; foo ‘y’ parameter
foo_localz      .BLKW 1         ; foo ‘z’ local
bar_rv          .BLKW 1         ; bar return value
bar_ra          .BLKW 1         ; bar return address
bar_paramw      .BLKW 1         ; bar ‘w’ parameter
```

- Code for foo() and bar() are somewhere else
- Function code call is similar to mixed solution

Possible Solution: Separate Code and Data

- Advantages:
  - Code can be marked ‘read only’
  - Conceptually easy to understand
  - Early Fortran used this scheme
  - Data persists through life of program
- Disadvantages:
  - Cannot handle recursion or parallel execution
  - Consumes resource for inactive functions

Real Solution: Run-time Stack

- Instead of allocating the space for local variables statically (i.e., in a fixed place in memory), the space is allocated once the function starts executing

- When the function returns to the caller, its space is reclaimed to be assigned later to another function

- If the function is called from itself, the new invocation of the function will get its own space that is distinct from its other currently active invocations

Run-time Stack: Stack frame

- Each function has a memory template where it stores its local variables, some bookkeeping information, and its parameter variables .This template is called its stack frame or activation record
- Whenever a function is called, its stack frame will be allocated somewhere in memory
- Because the calling pattern of functions naturally follows a stack-like pattern, this allocation and deallocation will follow the pushes and pops of a stack

Run-time Stack: stack-like nature of function calls

```
1  int main(void)
2  {
3     int a;
4     int b;
5
6     :
7     b = Watt(a);          // main calls Watt first
8     b = Volt(a, b);       // then calls Volt
9  }
10
11 int Watt(int a)
12 {
13    int w;
14
15    :
16    w = Volt(w, 10);      // Watt calls Volt
17
18    return w;
19 }
20
21 int Volt(int q; int r)
22 {
23   int k;
24   int m;
25
26   :
27   return k;
28 }
```

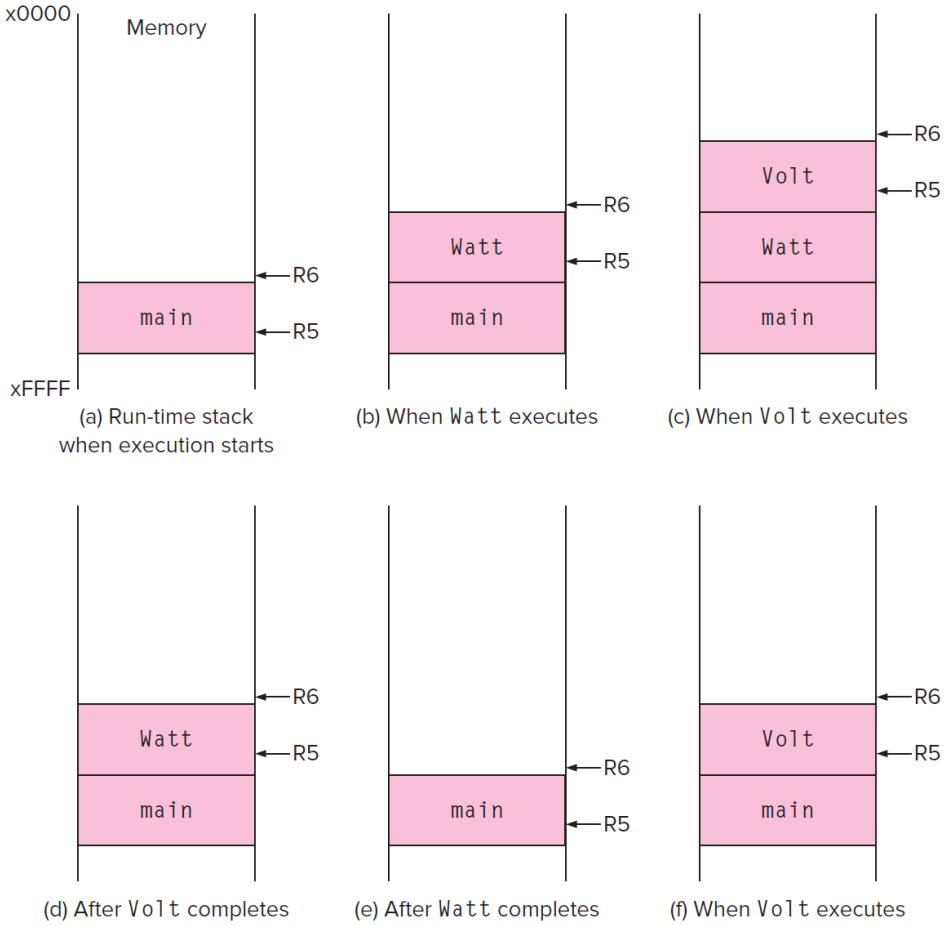


Figure 14.5 Several snapshots of the run-time stack while the program outlined in Figure 14.4 executes.

Run-time Stack: stack-like nature of function calls

- We need some easy way to manage the pushing and popping of stack frames
- For this, we will use R5 and R6
- R5 points to some internal location within the stack frame at the top of the stack—it may point to the base of the local variables for the currently executing function. We call it the frame pointer (FP)
- R6 always points to the very top of the stack. We call it the stack pointer (SP)

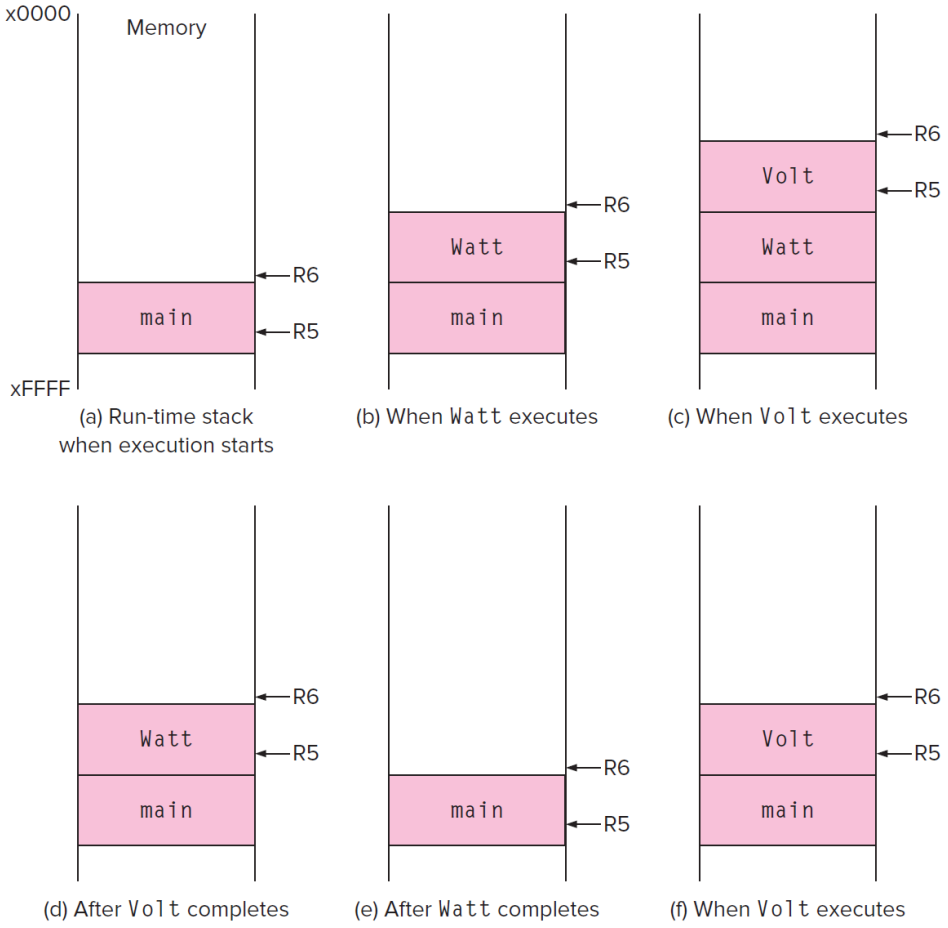


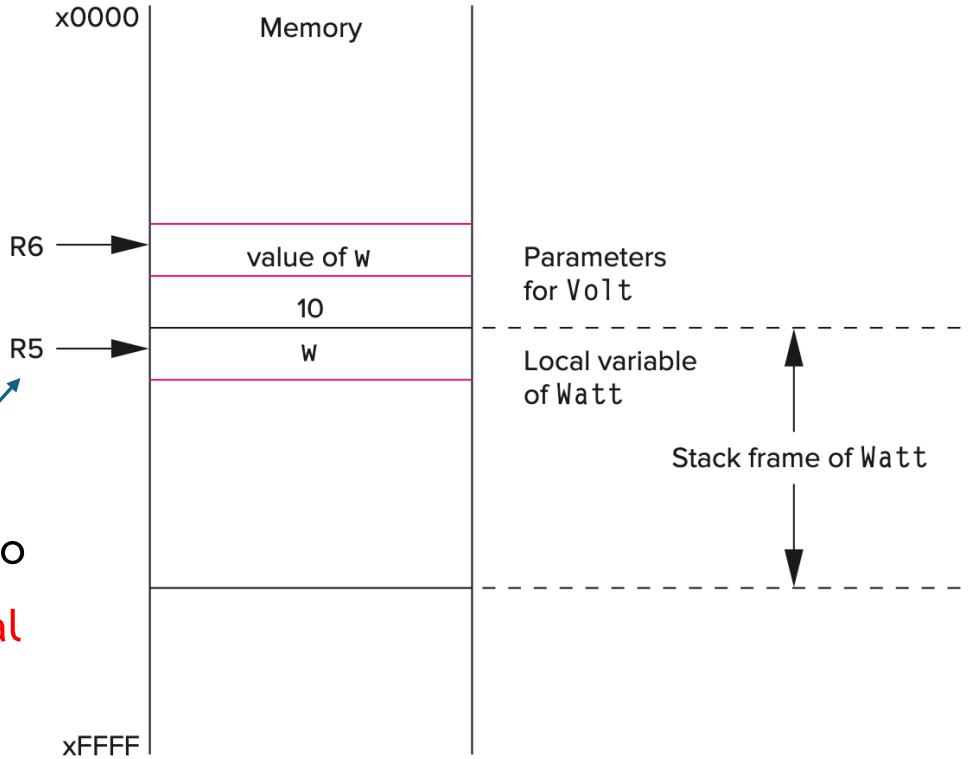
Figure 14.5 Several snapshots of the run-time stack while the program outlined in Figure 14.4 executes.

Run-time Stack: Stack frame

- Consider what has to happen in a function call:
  - Caller must pass parameters to the callee.
  - Caller must transfer control to the callee.
  - Caller need to allocate space for the return value.
  - Caller need to save the return address.
  - Callee requires space for local variables.
  - Callee must return control to the caller.
  - Callee need to save the frame pointer of the caller
- So, parameters, return value, return address, frame pointer, and local variables are stored on the stack.

The run-time stack after Watt pushes arguments to Volt

```
1  int main(void)
2  {
3     int a;
4     int b;
5
6     :
7     b = Watt(a);          // main calls Watt first
8     b = Volt(a, b);       // then calls Volt
9  }
10
11 int Watt(int a)
12 {
13    int w;
14
15    :
16    w = Volt(w, 10);      // Watt calls Volt
17
18    return w;
19 }
20
21 int Volt(int q; int r)
22 {
23   int k;
24   int m;
25
26   :
27   return k;
28 }
```



stack frame points to the base of the local variables for the currently executing function

Run-time Stack: an example

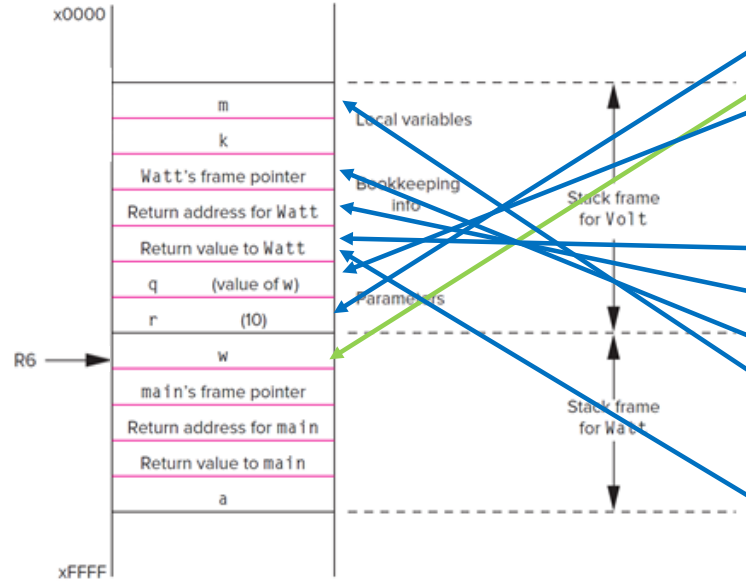


Figure 14.7 The run-time stack after the stack frame for `Volt` is pushed onto the stack.

```
1  Watt:
2          ...
3          AND R0, R0, #0    ; R0 <- 0
4          ADD R0, R0, #10   ; R0 <- 10
5          ADD R6, R6, #-1   ;
6          STR R0, R6, #0    ; Push 10 onto stack
7
8          LDR R0, R5, #0    ; Load w
9          ADD R6, R6, #-1   ;
10         STR R0, R6, #0    ; Push w
11         JSR Volt
12
13         LDR R0, R6, #0    ; Load the return value
14         STR R0, R5, #0    ; w = Volt(w, 10);
15         ADD R6, R6, #3    ; Pop return value and arguments
16         ...
17
18 Volt:   ADD R6, R6, #-1   ; Allocate spot for the return value
19
20         ADD R6, R6, #-1   ;
21         STR R7, R6, #0    ; Push R7 (Return address)
22
23         ADD R6, R6, #-1   ;
24         STR R5, R6, #0    ; Push R5 (Caller's frame pointer)
25
26         ADD R5, R6, #-1   ; Set frame pointer for Volt
27         ADD R6, R6, #-2   ; Allocate memory for Volt's local variables
28
29         ...               ; Volt performs its work
30
31         LDR R0, R5, #0    ; Load local variable k
32         STR R0, R5, #3    ; Write it in return value slot, which will always
33                           ; be at location R5 + 3
34
35         ADD R6, R5, #1    ; Pop local variables
36
37         LDR R5, R6, #0    ; Pop the frame pointer
38         ADD R6, R6, #1    ;
39
40         LDR R7, R6, #0    ; Pop the return address
41         ADD R6, R6, #1    ;
42         RET
```

Figure 14.8 The LC-3 code corresponding to a C function call and return.

```
1  int main(void)
2  {
3     int a;
4     int b;
5
6     :
7     b = Watt(a);          // main calls Watt first
8     b = Volt(a, b);       // then calls Volt
9  }
10
11 int Watt(int a)
12 {
13    int w;
14
15    :
16    w = Volt(w, 10);      // Watt calls Volt
17
18    return w;
19 }
20
21 int Volt(int q; int r)
22 {
23   int k;
24   int m;
25
26   :
27   return k;
28 }
```

Homework

Exercises:

14.1, 14.2, 14.6, 14.9